

[분산프로그래밍1 중간 레포트]

장찬영 60221341

장승완 60221314

이기현 60213037

<목차>

I. 서론

II. Use Case Diagram

III. Use Case Scenario

IV. Class Diagram

V. 결론

I. 서론

본 보고서는 분산프로그래밍1 수업을 수강하며 신동아화재 차세대 시스템 구축 제안요청서(RFP)를 기반으로 보험사 시스템의 요구사항을 분석하고 객체지향 설계 기법을 적용하여 시스템을 설계하는 것을 목적으로 한다. 이를 위해 유스케이스 다이어그램을 통해 시스템의 기능적 요구사항과 액터 간의 상호작용을 도출하고, 유스케이스 시나리오를 통해 각 기능의 세부 흐름을 명세하였으며, 클래스 다이어그램을 통해 시스템 내부의 데이터 구조와 클래스 간의 관계를 표현하였다. 분석 및 설계 범위는 RFP의 To-Be 프로세스 체계도를 기준으로 상품 개발, 언더라이팅(U/W), 계약 관리, 보상 처리 영역으로 설정하였다.

II . Use Case Diagram

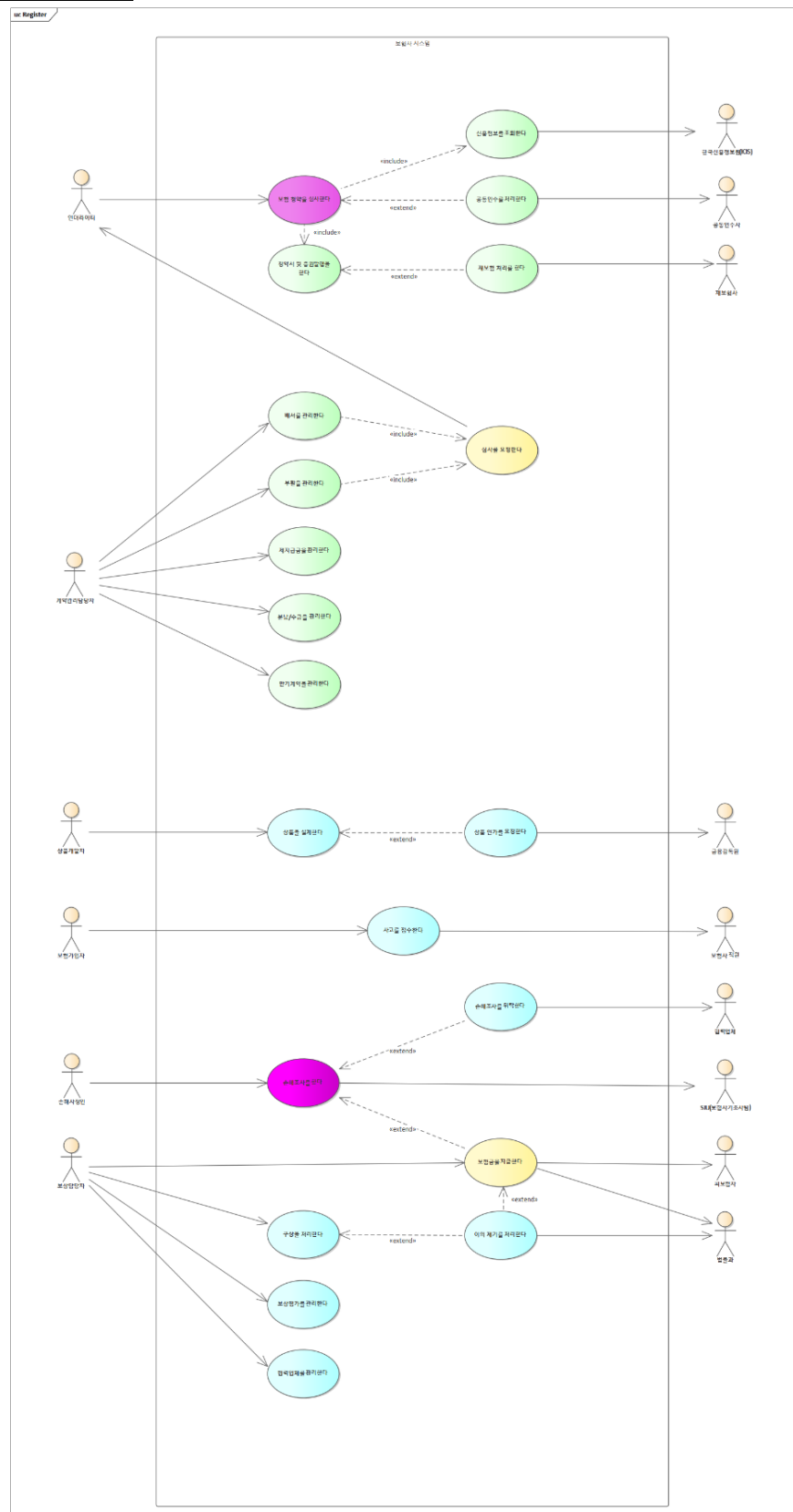


그림1: Use Case Diagram(유스케이스 다이어그램)

본 시스템의 범위는 PDF에서 제시된 To-Be 프로세스 체계도(P12)를 기준으로 설정하였다. 액터는 해당 유스케이스와 직접 상호작용하여 데이터를 입력·조회하거나, 외부 시스템으로부터 정보를 주고받는 주체를 기준으로 선정하였다. 이번 과제에서는 시스템을 사용하지 않는 액터 혹은 시스템에서 불필요한 액터들은 제외하였다.

1-1. 상품 개발 PDF상 흐름: 고객 니즈 파악 + 시장/경쟁 분석 → 보장내용 결정 → 보험요율 산출 → 상품 설계 → 인가 → 판매 지원 → 사후 평가

유스케이스(세부 업무)

상품기획: 전략 수립, 시장조사, 고객 니즈 분석, 개발안 수립
무엇을 만들지 결정하는 단계.

▶ 전문가들끼리 회의, 분석, 의사결정 하는 과정은 시스템에서 작동하는 것이 아니다. 그렇기에 유스케이스에서 제외하였다.

상품설계: 보장·판매 대상 결정, 요율 수집 및 시산(試算), 예상 손익분석, 가격 결정, 기초서류 작성, 요율 검증 의뢰
아이디어를 수치화 하는 과정.

▶ 시스템에서 다뤄야 할 내용이다. 특히 사람이 판단하는 행위가 아닌 시스템을 통해 수행되는 행위이므로 포함하였다.

상품인가: 상품 인가 품의, 상품 확정 (금융감독원 대상)
외부 승인 단계.

▶ 문서를 시스템에서 생성하고 제출하는 행위로 금융감독원이라는 외부 액터와 상호작용이 발생하는 지점이다. 시스템에서 다뤄야 할 내용이므로 포함하였다.

상품판매지원: 가입설계·청약·계약관리 관련 시스템 개발, 판매부 자료 제작, 상품교육
영업 준비 단계 혹은 사람 교육 단계

▶ 판매지원의 핵심인 상품교육은 사람이 수행하는 교육 행위로 상품개발자가 시스템을 통해 달성하는 목표가 아니기 때문에 제외하였다.

상품사후관리: 판매 실적 및 속성 분석, 문제점 파악 및 손익분석
출시 후 모니터링 단계

▶사후관리는 현 과제에서 필요 없다는 교수님의 피드백에 의해 삭제하였다

액터

상품개발자 - 시스템과 상호작용하는 내부 사용자. 상품설계, 인가요청 등 시스템에 뭔가를 입력하거나 조회하는 주체이다.

금융감독원 - 시스템 경계 밖에서 인가 결과(승인/반려)를 돌려주는 외부 액터이다. 업무 흐름상 승인/반려 결과가 시스템에 반영되기 때문에 액터로 포함하였다.

액터 선정 이유

상품개발 단계에서는 상품설계와 상품인가가 시스템을 통해 수행되는 핵심 업무이다. 상품 개발 단계에서 상품 구조를 설계하고 인가 요청을 수행하는 주체는 상품 개발자 이므로 액터로 선정하였다. 상품인가 과정은 금융감독원의 승인 여부에 따라 결과가 결정되므로, 시스템 외부에서 승인/반려 결과를 제공하는 금융감독원을 외부 액터로 선정하였다.

1-2 U/W (언더라이팅) "이 계약을 받아도 되는가"를 판단

PDF상 흐름: 인수정책 수립 → 청약 접수 → 인수심사 (자동/수동) → 승인/거절 → 계약 확정 → (재보험/공동인수) → 손해율 분석 → 정책 피드백 유스케이스(세부 업무)

유스케이스(세부 업무)

인수정책수립: 인수자료 수집, 재보험 분석, 손해율 분석, 인수지침 등록
계약을 어떤 기준으로 인수할지 결정하는 단계.

▶ 시장환경 분석, 손해율 예측 등은 전문가의 판단과 전략 수립 중심의 업무이다. 분석 후 계획을 수립하는 것은 사람이 수행한다고 가정하였다. 그렇게 해서 최종 인수 정책 결과는 커뮤니티 게시판 같은 내부 네트워크를 통해 공유한다고 가정하고, 기능으로 추가하지는 않았다.

인수심사: 청약 기반 자동심사, 진단심사, 특인심사, 일반심사, 이미지심사, 적부심사 및 증권 발행

실제 고객 또는 계약에 대해 위험도를 평가하고 인수 여부를 판단하는 단계.

▶ 심사 과정은 데이터 기반으로 자동화 및 시스템 처리된다. 자동심사 및 다양한 심사 유형은 시스템을 통해 수행되는 기능이므로 유스케이스에 포함하였다.

추가)심사의 경우들은 내부 시나리오 alternative flow로 별도로 작성하였다.

재보험처리: 재보험료 산출, 검증, 계상, 청산 및 등록

고위험 계약에 대해 리스크를 외부 보험사에 분산하는 단계.

▶ 재보험료 산출 및 처리 과정은 시스템을 통한 계산 및 데이터 처리가 핵심이므로 유스 케이스에 포함하였다.

공동인수: 공동인수 접수, 참여지분 및 보유액 통보, 출재 및 계상, 청산

여러 보험사가 하나의 계약을 분담하여 인수하는 단계.

▶ 공동인수 과정에서 지분 계산, 통보 및 정산은 시스템 기반으로 처리되는 업무이므로 유스 케이스에 포함하였다.

손해율관리: 손해율 데이터 수집, 분석, 평가 및 예측

인수 결과에 따른 손익을 분석하고 향후 인수정책에 반영하는 단계.

▶ 손해율 분석은 데이터 기반 분석 업무로 시스템에서 수행될 수 있으나, 정책 수립 단계와 마찬가지로 분석 결과를 활용하는 영역으로 판단하여 제외하였다.

액터

언더라이터 - 시스템과 상호작용하는 내부 사용자로, 청약 심사를 수행하고 인수 여부를 결정하기 위해 시스템을 활용하는 주체이다.

재보험사 - 외부 시스템으로, 재보험 처리 과정에서 리스크를 분산 받는 역할을 수행하는 액터이다.

공동인수사 - 외부 보험사로, 공동인수 계약에서 지분을 나누어 인수하는 주체이다.

한국 신용 정보원 - 외부 기관으로, 청약 심사 과정에서 필요한 신용정보를 제공하는 액터이다.

액터 선정 이유

U/W 단계에서는 보험청약을 심사하여 계약의 인수 여부를 결정하는 것이 핵심이다. 이때 시스템을 통해 데이터를 조회하고 심사 결과를 확정하는 중심 역할은 언더라이터이기에 액터로 선정하였다. 심사 과정에서는 외부 정보와 리스크 분산이 중요한 요소로 작용한다. 한국 신용 정보원은 사고이력 및 타사 계약정보를 제공하여 심사의 정확도를 높이는 역할을 하며, 재보험사와 공동인수사는 고위험 계약에 대한 리스크를 분산하는 데 직접적으로 관여하기에 외부 액터로 선정하였다.

1-3 계약관리

PDF상 흐름: 계약관리지침 수립 → 수금/분납 관리 → 만기계약 관리 → 제지급금 관리 → 부활 관리 → 배서 관리 → 우편물 관리 → 계약통계 관리

유스케이스(세부 업무)

계약관리지침수립: 자료수집, 지침개정 검토, 수립, 통보 및 교육, 통합관리 등 계약관리 업무의 기준과 운영 방식을 정의하는 단계.

▶ 전문가의 판단과 조직 내부 의사결정을 중심으로 수행되는 업무 이므로 유스케이스에서 제외하였다.

분납/수금관리: 수금대상추출, 자동이체, 지로 및 방문수금, 이관, 취소 및 미납 안내, 통합관리

보험료를 분할 납부하거나 수금하는 단계.

▶ 시스템을 통해 반복적으로 수행되는 데이터 처리 업무이므로 유스케이스에 포함하였다.

만기계약관리: 만기대상추출 및 안내, 안내장발송, 재계약여부 확인, 현황 평가, 통합관리 계약 만료 시점에서 계약을 종료하거나 갱신하는 단계.

▶ 만기 대상 조회, 안내장 발송 등은 시스템을 통해 처리되며 재계약을 시스템에 등록하므로 유스케이스에 포함하였다.

제지급금관리: 지급요청접수, 지급안내장 발송, 지급금산출 및 검증, 자동지급 및 지급처리, 통합관리

보험계약과 관련된 각종 지급금을 처리하는 단계.

▶ 지급금 산출 및 검증, 자동지급 처리 등은 시스템 기반 계산 및 처리 기능이 핵심이므로 유스케이스에 포함하였다.

부활관리: 부활대상추출, 등록, 자동심사, 진단심사, 특인심사, 일반심사, 이미지심사 실패된 계약을 다시 활성화하는 단계.

▶ 부활 대상 조회 및 심사 과정은 시스템을 통해 수행되며, 심사 기능은 자동화 및 데이터 기반 처리이므로 유스케이스에 포함하였다.

배서관리: 배서입력, 자동심사, 진단심사, 특인심사, 일반심사, 이미지심사

계약의 조건을 변경하는 단계.

▶ 계약 변경 정보 입력 및 심사 과정은 시스템을 통해 처리되며 조건 변경 시 위험도 재평가가 필요하므로 유스케이스에 포함하였다.

추가1) 부활, 배서 과정 중 인수 판단이 필요한 경우 <<extend>> [심사를 요청한다] 유스케이스를 수행한다.

추가2) 심사의 경우 U/W가 담당하기에 U/W 액터와 연결 시켰다

우편물관리: 우편물 통합관리, 발송 및 반송접수, 반송우편물 관리

약 관련 문서를 고객에게 전달하는 단계.

▶ 우편물 발송 및 반송 처리는 우편물 부서가 (사람들이) 수행하는 작업으로 가정하여 유스케이스에서 제외하였다.

계약통계관리: 기초 데이터추출, 마감통계, 특별계정, 효율통계, 평가 및 제공

계약 데이터를 기반으로 성과를 분석하고 평가하는 단계.

▶ 사후지원 관리와 마찬가지로 해당 과제에서 불필요하다 판단하고 삭제하였다. 동일하게 전문가가 직접 관리하는 것으로 가정하였다

액터

계약담당관리자 - 시스템과 상호작용하는 내부 사용자로 계약관리 업무를 수행하는 주체이다.

액터 선정 이유

계약관리 단계에서는 계약의 유지, 변경, 수금 및 지급 등 계약 이후 발생하는 모든 업무를 처리하는 것이 핵심이다.

해당 업무의 경우 계약담당관리자가 시스템을 통해 수행하므로 액터로 선정하였다

추가적으로 계약 관리 중 심사의 경우 U/W가 담당하기에 U/W 액터와 연결 시켰다

2-1 보상기획

PDF상 흐름: 보상전략 수립 → 보상운영 관리 → 보상평가 관리 → 협력업체 관리

유스케이스(세부 업무)

보상전략수립: 보상전반에 걸친 운용점검 분석결과, 보상통계 평가결과, 시장분석보고서, 협력업체 평가결과 등 회사 내부현황 분석자료와 업계 및 선진사례와의 이슈 및 Gap 분석등을 통한 장/단기 보상전략을 수립하고, 이러한 보상전략을 반영한 사업계획을 수립하는 업무이다.

보상 업무의 방향성과 기준을 설정하는 단계.

▶ 보상전략수립은 각종 자료와 분석 결과를 바탕으로 장·단기 전략 및 사업계획을 결정하는 업무이고 전문가의 판단과 조직 내부 의사결정을 통해 수행되는 영역이다. 시스템에서 작동하는 것이 아니라고 보고 유스케이스에서 제외하였다.

보상운용관리: 보상운용 기초자료 수집/분석, 동업사 현황 및 국내외 사례조사 등을 통해 보상교육, 인력운용, 업무평가, 업무지침, 시스템도입 등에 관한 세부 운용관리안을 수립/관리하고 운용점검 및 분석하는 업무이다.

보상 업무가 정상적으로 수행되도록 운영을 관리하는 단계.

▶ 실제 운용안 수립과 관리 판단은 사람이 수행하는 것으로 가정하고 유스케이스에서 제외하였다.

보상평가관리: 월별마감/기초통계를 집계 및 산출하고 손해액을 분석하는 보상 통계업무와 보상조직 및 직원에 대한 실적평가를 위한 기초자료를 수집/산출/분석/평가하는 업무이다.

보상 업무의 결과를 분석하고 평가하는 단계.

▶ 보상평가관리는 통계 집계와 손해액 분석은 시스템을 통해 진행하므로 유스케이스에 포함하였다.

협력업체관리: 병원, 정비공장, 법무법인, 손해사정업체, 신용정보회사, 현장출동 및 긴급출동 대행업체 등 협력업체의 보상처리 결과 및 분석을 통하여 협력업체를 선정, 관리, 평가하는 업무이다.

보상 업무에 필요한 외부 업체를 관리하는 단계.

▶ 협력업체를 등록하고 관리하는 것은 시스템을 통한 데이터 관리 업무이므로 유스케이스

에 포함하였다.

액터

보험 담당자: 보상처리의 핵심 업무를 수행하는 주체

액터 선정 이유

보상서비스 제공 및 운영을 위한 전체적인 방향을 설정하는 업무이다. 시스템을 통해 최종적인 보상 처리를 수행하는 중심 역할을 보험 담당자가 진행하기에 액터로 선정하였다

2-2 보상 처리

PDF상 흐름: 사고접수 → 손해조사 → 손해사정 → 보험금 지급 및 종결 → 소송 → 구상

유스케이스(세부 업무)

사고접수: 보상조직 및 콜센터의 사고접수, 모바일을 통한 현장출동과 긴급출동 지시, 사고 발생지역 및 피해자(물)의 상태(병원/정비공장 처리여부)에 따른 전산배당업무
보상처리의 시작 단계.

▶ 사고 정보(증권번호, 사고 일시, 사고 경위, 피해 내용)를 입력하고, 관련 서류를 업로드하여 시스템에 등록하는 데이터 입력 중심의 업무이다. 접수 완료 시 사고번호 생성 및 상태 관리(현장 조사 필요 등)가 수행되므로 시스템의 핵심 기능으로 판단하여 유스케이스에 포함하였다.

손해조사: 손해사정업체 및 자체조사를 통한 조사보고서 작성 및 면/부책 판단, 지급준비금을 산출/적립하는 업무

사고에 대한 사실 확인 및 손해 여부를 판단하는 단계.

▶ 결과 입력, 보고서 작성, 지급준비금 산출 등 결과를 기록·관리하는 기능이 시스템에서 수행되므로 유스케이스에 포함하였다.

손해사정: 병원/정비공장/고객으로부터 청구된 보험금 접수, 손해액 및 결정보험금을 산출하는 업무

손해액을 산정하고 지급 금액을 결정하는 단계.

▶손해조사 안에서 조사 + 판단 + 일부 산정까지 수행하기 때문에 손해조사로 합쳤다.

지급 및 종결: 지급품의서를 작성하고 책임자의 결재를 득하여 보험금을 결정/지급/종결 처리하는 업무

보험금 지급 및 보상처리 종료 단계.

▶보험금 지급 금액 확정, 지급 처리, 상태 변경 등을 시스템에서 처리하고 결과를 반영해야 하는 보험의 핵심 업무이므로 유스케이스에 포함하였다

소송: 소송과 관련한 대리인 선정, 소송진행 및 항소여부 결정 등의 관리업무

분쟁 발생 시 법적 절차를 수행하는 단계.

▶소송 자체가 법적 판단, 대리인 선정, 항소 여부 결정 등 사람 중심의 의사결정 업무 이기에 '이의 제기 하다'라는 유스케이스를 통해 법률로 이관 하였다.

구상: 잔존물 및 제3 자에 대한 손해배상청구권을 대위하여 구상처리하는 업무(가압류, 소송, 강제집행, 채권추심 등)

보험금 지급 이후 비용을 회수하는 단계.

▶보험금 지급 이후 발생하는 후속 처리 과정으로 시스템적 처리 요소가 많기 때문에 별도 유스케이스로 반영하였다.

액터

손해 사정인 - 사고에 대한 손해조사를 수행하고 조사보고서를 작성하며, 손해 여부 판단에 필요한 정보를 제공하는 주체이다.

보험 담당자 - 보험금 지급, 구상 처리, 이의제기 처리 등 보상처리의 전반적인 업무를 수행하는 내부 사용자이다.

협력 업체 - 손해조사, 수리, 치료 등 보상처리와 관련된 업무를 위탁받아 수행하는 외부 업체이다.

보험사기 조사팀 - 보험사기 의심 사례에 대해 추가적인 조사 및 분석을 수행하는 특수 조직이다.

피보험자 - 보험계약의 보장 대상자로, 보험금 지급의 직접적인 대상이 되는 주체이다.

법률과 - 이의제기 및 소송 등 분쟁 발생 시 법적 절차를 수행하는 외부 기관이다.

액터로 뽑은 이유

보상처리 단계에서는 사고 발생 이후 사고접수, 손해조사, 손해사정, 보험금 지급 및 종결까지의 일련의 과정을 통해 최종적으로 보상이 이루어지는 것이 핵심이다. 이때 시스템을 통해 사고를 접수하고 정보를 입력하는 역할은 보험가입자이기에 액터로 선정하였다. 사고조사 결과를 입력하고 판단하는 중심 역할은 손해사정인이다. 또한 보험금 지급 및 종결, 구상 처리 등 최종 보상 결과를 시스템에 반영하는 주체는 보상 담당자이므로 내부 액터로 선정하였다. 보상처리 과정에서는 외부 기관과의 연계도 중요하다. 협력업체는 손해조사 위탁을 통해 조사 결과를 제공한다. 보험사기 조사팀은 특수한 경우 추가 조사를 수행하여 보상 판단에 영향을 미친다. 피보험자는 보험금 지급의 직접적인 대상이며 이의제기 등의 행위를 수행할 수 있다. 법률기관은 분쟁 발생 시 소송 및 법적 절차를 통해 보상 결과에 영향을 미친다. 따라서 이들 외부 주체는 보상처리 결과에 직접적인 영향을 미치므로 외부 액터로 선정하였다.

<<include>> <<extend>> 관계 구분 (+ 유스케이스상 시나리오 흐름)

1-1. 상품 개발 부분

유스케이스 상 흐름: 상품설계한다 → 상품인가를 요청한다→ (승인 시) 상품을 확정한다 → 이후 판매 단계로 넘어간다.

상품 설계하다 and 상품 인가를 요청한다: <<extend>> 관계

상품 설계가 완료되면 금융감독원에게 상품 인가를 요청해야 한다.

이때 상품 설계가 완료되었다고 해서 무조건 상품 인가를 요청하는 것은 아니다. 설계는 먼저 하고 수정을 하지 않아 인가를 나중에 요청하던가, 혹은 상품 설계가 완료되어도 상품성이 없거나 상급자로부터 반려되어 취소되는 경우도 존재한다. 이처럼 설계 완료 후 필요시에 (확정시) 상품 인가를 요청하는 경우이기에 <<extend>> 관계로 선정하였다.

1-2. U/W (언더라이팅) 부분

유스케이스 상 흐름: 언더라이터가 신용 정보를 조회한다 -> 보험청약을 심사한다 -> <<extend>> 공동 인수 처리한다 -> 심사가 완료 된다 -> <<extend>>재보험 처리한다 -> 청약서 및 증권 발행을 한다

보험 청약을 심사한다 and 청약서 및 증권 발행을 실시한다: <<include>> 관계

청약 심사 및 절차가 완료된 이후 증권(공모주) 발행은 필수이다. 그렇기 때문에 <<include>> 관계로 선정하였다

보험 청약을 심사한다 and 신용 정보를 조회한다: <<include>> 관계

보험 청약을 심사하기 위해서는 반드시 신용 정보가 필요하다고 SW 설계를 하였다. 필수 절차 이므로 <<include>> 관계로 선정하였다.

보험청약을 심사한다 and 공동인수 처리한다: <<extend>> 관계

모든 청약이 공동인수로 처리되는 것이 아닌 위험이 크거나 보험가입금액이 커서 단독 인수가 부담되는 경우에만 공동인수가 필요하기 때문에 <<extend>> 관계로 선정하였다.

청약서 및 증권 발행한다 and 재보험 처리한다: <<extend>> 관계

재보험은 자사 보유한도를 초과하거나 위험 분산이 필요한 경우에만 수행되기 때문에 <<extend>> 관계로 선정하였다.

1-3 계약 관리 부분

유스케이스 상 흐름: 계약관리담당자가 각각의 계약관리 업무를 수행한다. 계약 관리 업무는 다음과 같다.

- 배서 관리 -> <<include>> 심사를 요청한다 (U/W 주체)
- 부활 관리 -> <<include>> 심사를 요청한다 (U/W 주체)
- 제지급금 관리
- 분납/수금 관리
- 만기계약 관리

배서를 관리한다 and 심사를 요청한다: <<include>> 관계

부활을 관리한다 and 심사를 요청한다: <<include>> 관계

배서와 부활 관계는 다른 세부 업무들과 특징이 다르다. 제지급금관리, 분납/수금관리, 만기계약관리의 경우 이미 확정된 계약을 기반으로 데이터 처리/관리만 수행하기 때문에 위험 판단 필요가 상대적으로 적다.

반면 배서와 부활 관계의 경우 보험가입금액 변경, 특약 추가/삭제, 수익자 변경 등 보장내용이 바뀔 수 있기 때문에 보험사가 부담할 위험이 달라진다. 그렇기에 U/W의 인수 판단이 다시 필요 하다 (이로 인해 U/W와 심사하다를 요청하다가 연결된다.)

따라서 심사 과정이 배서와 부활 관리할 때 반드시 필요하다 판단하였고 이로 인해 <<include>> 관계로 선정하였다.

2-1 보상기획

유스케이스 상 흐름: 보험 담당자가 보상 평가를 관리하거나 혹은 협력 업체를 관리한다.

보상기획 영역에서는 유스케이스 간의 관계(<<include>>, <<extend>>)는 존재하지 않는다.

기존 보상전략수립과 보상운용관리는 시스템에서 이루어지지 않으므로 유스케이스에 작성하지 않았다. 그 이후에 진행되는 보상 평가 관리 과정은 이후에 협력 업체를 관리한다는 과정이 반드시 함께 수행되거나 특정 조건에서 확장되는 관계가 아니므로 <<include>> 또는 <<extend>> 관계로 표현하지 않았다.

즉 보상평가관리와 협력업체관리는 서로 독립적인 관리 업무로 판단하여 각각 별도의 유스케이스로 구성하였다.

2-2 보상 처리

유스케이스 상 흐름: 사고를 접수 한다 -> 손해조사를 한다 -> 보험금을 지불한다. 이때 상황에 따라 이의제기, 구상, 소송 단계로 넘어갈 수 있다. 손해조사를 위탁한다는 손해조사 과정 중 외부 협력업체나 손해사정업체의 조사가 필요한 경우 수행된다

손해조사를 한다 and 손해조사를 위탁한다: <<extend>> 관계

손해 조사가 진행될 때 모든 사고를 외부에 맡기지 않는다. 기본적으로는 내부 조사를 수행하는 것을 원칙으로 하지만 대형 사고, 전문 필요 같은 외부 위탁이 필요 할경우 위탁이 진행되므로 <<extend>> 관계로 선정하였다.

손해조사를 한다 and 보험금을 지급한다: <<extend>> 관계

보험금 지급은 손해조사 결과에 따라 지급 가능 여부가 결정되는 것이지, 조사가 된다고 보험금을 지급하는 것이 아니다. 조건에 해당될 때만 보험금을 지급한다. 따라서 보험금 지급은 모든 손해조사 이후 반드시 수행되는 단계가 아닌 조건부로 수행되는 업무이기 때문에 <<extend>> 관계로 선정하였다.

보험금을 지급한다 and 이의제기를 처리한다: <<extend>> 관계

이의제기 처리는 보험금 지급 이후 고객의 이의가 있는 경우에만 수행되는 조건부 업무이므로 <<extend>> 관계로 선정하였다.

이의제기를 처리한다 and 구상을 처리한다: <<extend>> 관계

제 3자가 구상처리를 인정하지 못하는 경우에 이의제기를 할 수 있다. 하지만 그렇다고 해서 이의제기를 항상 하는 것은 아니므로 <<extend>> 관계로 선정하였다.

III. Use Case Scenario

보험청약을 심사한다	
액터	언더라이터
사전조건	✓ 청약서가 시스템에 접수되어 있어야 한다. ✓ 인수지침이 시스템에 등록되어 있어야 한다.
사후조건	➢ 심사 결과가 DB에 저장되고 청약서 및 증권이 발행된다. ➢ <<include>> [청약서 및 증권발행을 한다] 시나리오를 수행한다.
Basic Path	
1.	언더라이터는 피보험자 정보 (이름, 생년월일, 주민등록번호, 차량번호)를 입력한다.
2.	시스템은 자사 DB에서의 피보험자의 과거 U/W이력(날짜, 이름, 주민등록번호, 성별, 나이, 직업, 연 소득, 차량기종, 차량번호, 과거질병이력, 투약여부, 수술이력, 가족력, 흡연여부, 음주량, BMI)과 계약정보(증권번호, 계약상태, 청약일, 승낙일, 보험기간, 납입기간, 납입주기, 계약자정보, 피보험자정보, 수익자정보, 상품명, 주계약내용, 특약목록, 보험가입금액, 보험료, 납입일, 납입금액, 미납여부, 자동이체정보)를 화면에 출력한다. (A1)

3.	언더라이터는 '신용정보 조회' 버튼을 누른다. <<include>> [신용정보를 조회한다] 시나리오를 수행한다.
4.	시스템은 타사 계약정보(보험사명, 증권번호, 상품명, 계약상태, 보험기간, 보험가입금액, 보험료)를 화면에 출력한다. (A2)
5.	언더라이터는 '심사점수 계산 및 보고서 출력' 버튼을 누른다.
6.	시스템은 수집한 데이터와 현재 적용중인 인수지침 및 U/W Factor를 기반으로 Rule Based 엔진을 통해 자동심사를 실행하고, U/W Scoring 산출 결과와 추천등급(승인/할증/거절), 공동인수 추천 여부를 보고서 형태(총점, 항목별 점수, 감점 사유, 공동인수 추천 기준 값)로 화면에 출력한다. (A3)
7.	언더라이터는 심사 결과를 토대로 공동인수 처리를 결정한다. (공동인수 처리가 필요할 시 <<extend>> [공동인수를 처리한다] 시나리오를 수행한다.)
8.	언더라이터는 사원번호를 입력한 후 최종 심사결과(승인/할증/거절) 버튼을 누른다.
9.	시스템은 심사자 정보(이름, 부서, 사원번호), 피보험자 정보(날짜, 이름, 주민등록번호, 성별, 나이, 직업, 연 소득, 차량기종, 차량번호, 과거질병이력, 투약여부, 수술이력, 가족력, 흡연여부, 음주량, BMI), 심사결과(승인/할증/거절)를 문서 형태로 화면에 출력한 후 자사 DB에 저장한다. (A4) (E1)
Alternate	
A1. 자사 신규 고객이거나 갱신해야 할 정보가 있을 시	
A1-1. 자사 신규 고객일 시	
2.	시스템은 "자사 과거 U/W 이력 및 기존 계약 없음" 메시지를 화면에 출력한다.
3.	언더라이터는 신규 고객 데이터 작성 버튼을 누른다.
4.	시스템은 빈 U/W 정보 입력 폼을 화면에 출력한다.
5.	언더라이터는 신규 고객 정보(이름, 나이, 성별, 주민등록번호, 차량번호, 직업, 연소득, 과거질병이력, 투약여부, 수술이력, 가족력, 흡연여부, 음주량, BMI)를 입력한다.
6.	시스템은 추가된 정보를 자사 DB에 반영한다. (Basic Path 2번으로 복귀한다.)
A1-2. 갱신해야 할 정보가 있을 시	
1.	언더라이터는 변경이 필요한 항목(직업, 연소득, 투약여부, 수술이력, 흡연여부, 음주량, BMI, 차량기종, 차량번호)을 수정 입력한다.
2.	시스템은 수정된 정보를 자사 DB에 반영한다. (Basic Path 2번으로 복귀한다.)
A2. 타사 계약이 존재하는 경우	

2.	시스템은 타사 계약 정보를 보고서 형태로 출력한다.
3.	언더라이터는 심사 반영 버튼을 누른다.
4.	시스템은 자사 지침에 따라 점수로 변환하여 심사 반영한다. (Basic Path 4번으로 복귀한다.)
A3. 자동심사로 판단 불가능한 경우	
2.	시스템은 자동심사 불가 항목과 추가 심사 유형(진단심사/특인심사/일반심사/이미지심사/적부심사)을 화면에 출력한다.
3.	언더라이터는 해당 심사 유형을 선택한다. (A3-1, A3-2, A3-3, A3-4, A3-5)
A3-1. 진단심사인 경우	
2.	시스템은 진단심사 입력 품(진단명, 진단코드, 진단일, 담당의사, 진단결과, 진단서 첨부)을 화면에 출력한다.
3.	언더라이터는 진단심사 결과를 입력한다.
4.	시스템은 진단심사 결과를 점수로 변환하여 심사 반영한다. (Basic Path 6번으로 복귀한다.)
A3-2. 특인심사인 경우	
2.	시스템은 특인심사 입력 품(특인사유, 특인조건, 할증율, 부담보항목, 특인승인자)을 화면에 출력한다.
3.	언더라이터는 특인심사 결과를 입력한다.
4.	시스템은 특인심사 결과를 점수로 변환하여 심사 반영한다. (Basic Path 6번으로 복귀한다.)
A3-3. 일반심사인 경우	
2.	시스템은 일반심사 입력 품(심사항목, 심사결과, 심사의견, 참고자료 첨부)을 화면에 출력한다.
3.	언더라이터는 일반심사 결과를 입력한다.
4.	시스템은 일반심사 결과를 점수로 변환하여 심사 반영한다. (Basic Path 6번으로 복귀한다.)
A3-4. 이미지심사인 경우	
2.	시스템은 이미지심사 화면(청약서 이미지, 진단서 이미지, 기타 첨부서류 이미지)을 출력한다.
3.	언더라이터는 심사결과(이상 없음/이상 있음)와 심사의견을 입력한다.
4.	시스템은 이미지심사 결과를 점수로 변환하여 심사 반영한다. (Basic Path 6번으로 복귀한다.)

A3-5. 적부심사인 경우	
2.	시스템은 적부심사 입력 폼(계약적합성 항목, 적부결과, 적부의견, 현장조사 결과)을 화면에 출력한다
3.	언더라이터는 적부심사 결과를 입력한다
4.	시스템은 적부심사 결과를 점수로 변환하여 심사에 반영한다 (Basic Path 6번으로 복귀한다.)
A4. 심사결과가 적합이 아닌 경우	
A4-1. 거절인 경우	
2.	시스템은 거절사유를 문서에 추가로 출력한다.
A4-2. 할증인 경우	
2.	시스템은 할증조건을 문서에 추가로 출력한다.
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	언더라이터는 '다시 시도' 버튼을 누른다.
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

신용정보를 조회한다	
액터	언더라이터, 한국신용정보원(ICIS)
사전조건	✓ 유스케이스 시나리오 [보험청약을 심사한다]의 Basic Path 3번에서 시작되었다.
사후조건	➤ 타사 계약정보 및 사고이력이 심사에 반영된다.
Basic Path	
2.	시스템은 ICIS API를 호출하여 피보험자의 사고 이력(사고접수번호, 접수일, 사고일시, 사고장소, 사고유형, 진단명, 진단코드, 치료내용, 입원기간, 수술 여부, 청구금액, 인정금액, 지급일)을 화면에 출력한다. (A1) (E1)
3.	언더라이터는 '타사 계약 조회' 버튼을 누른다.
4.	시스템은 ICIS API를 호출하여 타사 계약정보(보험사명, 증권번호, 상품명,

	계약상태, 보험기간, 보험가입금액, 보험료)를 화면에 출력한다. ([보험청약을 심사한다]의 Basic Path 4번으로 복귀한다.)
Alternate	
A1. 사고 이력이 존재하는 경우	
2.	시스템은 사고 정보를 보고서 형태로 출력한다.
3.	언더라이터는 심사 반영 버튼을 누른다.
4.	시스템은 자사 지침에 따라 점수로 변환하여 심사 반영한다. (Basic Path 3번으로 복귀한다.)
Exception	
E1. ICIS API가 응답하지 않는 경우	
2.	시스템은 "ICIS API가 응답하지 않아 정보를 가져오는 것을 실패했습니다."를 출력한다.
3.	언더라이터는 '임시저장' 버튼을 누른다.
4.	시스템은 현재 상태를 자사 DB에 임시저장 후 시스템을 종료한다.
➤ 시스템이 종료된다.	

청약서 및 증권발행을 한다	
액터	언더라이터
사전조건	✓ 유스케이스 시나리오 [보험청약을 심사한다]가 완료된 상태이다.
사후조건	➤ 청약번호의 상태가 '심사 완료'로 변경된 후 미심사 청약 목록에서 제거된다.
Basic Path	
2.	시스템은 심사 완료된 청약 정보(청약번호, 피보험자 정보, 상품명, 보험가입금액, 보험료, 심사 결과, 적용 조건)를 청약서 확정 화면에 출력한다.
3.	언더라이터는 청약서 내용(계약자 정보, 피보험자 정보, 수익자 정보, 보장내용, 보험료, 특약 사항)을 '승인' 버튼을 눌러 승인한다. (A1)
4.	시스템은 증권번호를 자동 채번하고 보험증권 데이터(증권번호, 계약일, 보장 개시일, 보장 내용, 보험료, 약관 정보)를 화면에 출력한다.
5.	언더라이터는 '발행 승인' 버튼을 누른다.
6.	시스템은 보험증권 발행 처리결과 ("청약서 확정 및 증권발행 완료 (증권번호: XXXX-XXXX-XXXX)")를 출력하고, 계약 정보(증권번호, 청약번호, 계약일,

	보장 개시일, 보장 만료일, 계약 상태, 계약자 정보, 피보험자 정보, 수익자 정보, 상품코드, 보험가입금액, 보험료, 납입주기, 보장 범위, 특약 사항, 적용 조건, 약관 버전)를 자사 DB에 저장한다. (재보험 처리가 필요할 시 <<extend>> [재보험 처리를 한다] 시나리오를 수행한다.) (E1)
7.	언더라이터는 '보험증권 발행 완료' 버튼을 누른다.
8.	시스템은 청약번호의 상태를 '심사 완료'로 변경한 후 미심사 청약 목록에서 제거한다.
Alternate	
A1. 청약서 내용에 오류가 있는 경우	
1.	언더라이터는 '청약서 수정' 버튼을 누른다.
2.	시스템은 수정 가능한 문서의 형태로 전환한다.
3.	언더라이터는 잘못된 정보를 변경 후 '수정 완료' 버튼을 누른다.
4.	시스템은 시스템 관리자에게 오류 리포트를 전달하고 문서를 업데이트한다. (Basic Path 3번으로 복귀한다.)
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	언더라이터는 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 7번으로 복귀한다.)
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

공동인수를 처리한다	
액터	언더라이터, 공동인수사
사전조건	✓ 유스케이스 시나리오 [보험청약을 심사한다] Basic Path 7번에서 공동인수 처리 결정 시 시작된다. ✓ 공동인수 가능한 보험사 목록이 시스템에 등록되어 있어야 한다.
사후조건	➤ 공동인수 접수 정보(참여지분, 보유액)가 DB에 저장되고 출재 및 계상 처리가 완료된다. ➤ [보험청약을 심사한다]의 Basic Path 8번으로 복귀한다.
Basic Path	

1.	언더라이터는 공동인수 신청 화면에서 공동인수사(보험사명)를 선택하고, 자사 보유 지분율(%)과 각 공동인수사 지분율(%)을 입력한다. (A1)
2.	시스템은 보험가입금액 기준으로 자사 보유액 및 각 공동인수사 보유액을 자동 산출하여 화면에 출력한다.
3.	언더라이터는 '공동인수 접수' 버튼을 누른다.
4.	시스템은 각 공동인수사에 공동인수 참여 요청 정보(청약번호, 피보험자 정보, 보험가입금액, 참여 요청 지분율, 보유액, 보험료 배분액)를 전송한다. (E1)
5.	시스템은 각 공동인수사의 승인 완료를 확인한 후, 공동인수 접수 정보(청약번호, 자사 보유 지분율 및 보유액, 공동인수사별 보험사명·지분율·보유액, 접수일시)를 DB에 저장한다. (A2)
6.	시스템은 자사 부담 보험료를 출재 처리하고 각 공동인수사별 보험료 배분을 계상한다. 화면에 공동인수 처리 완료 메시지("공동인수 접수 완료 (청약번호: XXXX)")를 출력한다. ([보험청약을 심사한다]의 Basic Path 8번으로 복귀한다.)
Alternate	
A1. 추천 공동인수사 목록에 없는 보험사를 직접 지정하는 경우	
1.	언더라이터는 '직접 지정' 버튼을 누른다.
2.	시스템은 등록된 전체 공동인수 가능 보험사 목록(보험사명, 공동인수 가능 여부, 최대 인수 가능 지분율)을 화면에 출력한다.
3.	언더라이터는 목록에서 공동인수사를 선택하여 지분율을 입력한다. (Basic Path 3번으로 복귀한다.)
A2. 공동인수사가 참여를 거절하는 경우	
2.	시스템은 거절 사유(거절 보험사명, 거절 사유)를 화면에 출력하고 언더라이터에게 알림을 발송한다.
3.	언더라이터는 대체 공동인수사를 선택하거나 지분 구조를 재조정하여 입력한다. (Basic Path 2번으로 복귀한다.)
Exception	
E1. 공동인수사 시스템 연결이 실패하는 경우	
2.	시스템은 "공동인수사 시스템 연결에 실패하였습니다. 잠시 후 다시 시도해주세요." 오류 메시지를 화면에 출력한다.
3.	언더라이터는 '임시저장' 버튼을 누른다.
4.	시스템은 현재 공동인수 접수 상태를 자사 DB에 임시저장 후 시스템을 종료한다.

➤ 시스템이 종료된다.

재보험 처리를 한다	
액터	언더라이터, 재보험사
사전조건	✓ 유스케이스 시나리오 [보험청약을 심사한다]가 완료된 상태이다. ✓ 유스케이스 시나리오 [청약서 및 증권발행을 한다]의 Basic Path 6 번에서 시작되었다. ✓ 보험가입금액이 자사 보유한도를 초과하는 계약으로 파악된 상태이다.
사후조건	➤ 재보험 처리 결과가 시스템에 저장되고 계약 상태가 재보험처리완료로 갱신된다. ➤ [청약서 및 증권발행을 한다]의 Basic Path 7번으로 복귀한다.
Basic Path	
1.	언더라이터는 재보험 처리 대상 계약(계약번호, 상품유형, 보험금액 기준)을 선택한다.
2.	시스템은 계약 정보 및 위험 정보(보험금액, 위험등급, 손해율 정보, 인수 조건)를 조회하여 화면에 출력한다. (A1)
3.	언더라이터는 재보험 조건(재보험 방식, 재보험 비율, 재보험사)을 설정한다.
4.	시스템은 재보험사에 요청 정보(계약번호, 보험금액, 위험등급, 재보험 조건)를 전송한다.
5.	시스템은 재보험사로부터 인수 여부 및 조건(인수가능여부, 재보험비율, 재보험료, 조건부 인수사항)을 수신하여 화면에 출력한다. (E1)
6.	언더라이터는 '재보험 조건 검증 확인' 버튼을 누른다. (A2)
7.	시스템은 재보험 금액 및 분담 비율을 계산하고 재보험료를 회계장부에 계상한 후 계상 결과(재보험료, 출재비율, 보유비율, 계상금액, 계상일자, 계정과목)를 화면에 출력한다.
8.	언더라이터는 재보험 적용 내용을 확정하고 '저장' 버튼을 누른다. (A3)
9.	시스템은 재보험 처리 결과(계약번호, 재보험사, 재보험 비율, 재보험료, 계상일자)를 자사 DB에 저장하고 저장 완료 화면을 출력한다.
10.	언더라이터는 '청산 일정 등록' 버튼을 누른다.
11.	시스템은 청산 일정 입력 화면(청산예정일, 청산금액, 청산방식)을 출력한다.
12.	언더라이터는 청산 정보(청산예정일, 청산금액, 청산방식)를 입력한다.
13.	시스템은 청산 정보를 자사 DB에 저장하고 계약 상태를 '재보험처리완료'로

	갱신한 후 처리 완료 화면(계약번호, 재보험사, 재보험료, 청산예정일, 처리 완료일시)을 출력한다. ([청약서 및 증권발행을 한다]의 Basic Path 7번으로 복귀한다.)
Alternate	
A1. 재보험 대상이 아닌 경우	
2.	시스템은 “재보험 적용 대상이 아님” 안내 화면(판단 근거, 보유한도 정보)을 출력한다.
3.	언더라이터는 ‘결과 상세’ 버튼을 누른다.
4.	시스템은 재보험 대상 제외 근거 정보(보험가입금액, 자사 보유한도, 위험등급)를 출력한다.
5.	언더라이터는 ‘확인’ 버튼을 누른다.
➤ 시스템이 종료된다.	
A2. 재보험 조건을 수정할 경우	
1.	언더라이터는 재보험 비율 또는 조건을 수정한다. (input)
2.	시스템은 수정된 조건으로 재보험사에 재요청하고 수신된 결과를 화면에 출력한다. (output) (Basic Path 6번으로 복귀한다.)
Exception	
E1. 재보험사 응답 실패인 경우	
2.	시스템은 재보험사 응답 실패 화면(실패 사유, 재시도 안내)을 출력한다.
3.	언더라이터는 ‘확인’ 버튼을 누른다.
4.	시스템은 관리자에게 오류를 통보하고 현재까지 입력된 재보험 처리 정보를 임시저장한 후 임시저장 완료 화면(계약번호, 임시저장일시)을 출력하고 종료한다.
➤ 시스템이 종료된다.	

배서를 관리한다	
액터	계약관리담당자
사전조건	✓ 계약자 또는 피보험자로부터 계약 내용 변경 요청이 접수된 상태이다.
사후조건	➤ 배서 내용(증권번호, 배서유형, 변경 전 내용, 변경 후 내용, 변경사유, 심사결과, 처리일시)이 자사 DB에 저장되고 계약정보가 갱신된

	다.
Basic Path	
1.	계약관리담당자는 배서 신청 정보 (피보험자 이름, 주민등록번호, 증권번호, 배서유형, 보험가입금액, 납입주기, 특약추가/삭제)를 입력한다
2.	시스템은 해당 증권번호로 계약정보(계약상태, 보험기간, 보험가입금액, 보험 료, 납입주기, 특약목록)를 조회하여 화면에 출력한다 (A1)
3.	계약관리담당자는 변경할 배서항목(보험가입금액, 납입주기, 특약추가/삭제) 을 입력하고 배서신청 버튼을 누른다 (A2)
4.	시스템은 배서 신청 내용(배서유형, 변경 전 내용, 변경 후 내용, 변경사유, 신청일시)과 심사요청 버튼을 화면에 출력한다
5.	계약관리담당자는 심사요청 버튼을 누른다. <<include>> [심사를 요청한다] 시나리오를 진행한다.
6.	시스템은 심사 결과(승인/할증/거절)를 화면에 출력한다
7.	계약관리담당자는 확인 버튼을 누른다
8.	시스템은 배서 내용(증권번호, 배서유형, 변경 전 내용, 변경 후 내용, 변경 사유, 심사결과, 처리일시)을 자사 DB에 저장하고 계약정보를 갱신한다 (E1)
Alternate	
A1. 계약이 실효/해지/만기 상태인 경우	
2.	시스템은 "유효하지 않은 계약입니다" 메시지와 계약상태(실효일/해지일/만 기일)를 화면에 출력한다
3.	계약관리담당자는 확인 버튼을 누른다.
➤ 시스템이 종료된다.	
A2. 심사가 필요 없는 배서인 경우	
2.	시스템은 배서 신청 내용(배서유형, 변경 전 내용, 변경 후 내용, 변경사유, 신청일시)을 화면에 출력한다
3.	계약관리담당자는 확인 버튼을 누른다
4.	시스템은 배서 내용(증권번호, 배서유형, 변경 전 내용, 변경 후 내용, 변경 사유, 처리일시)을 자사 DB에 저장하고 계약정보를 갱신한다.
➤ 시스템이 종료된다.	
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다

3.	계약관리담당자는 다시 시도 버튼을 누른다
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다
➤ 시스템이 종료된다.	

부활을 관리한다	
액터	계약관리담당자
사전조건	✓ 피보험자의 계약 정보가 시스템에 등록되어 있어야 한다. ✓ 미납보험료 납입 확인이 완료된 상태이어야 한다.
사후조건	➤ 부활 처리 결과(증권번호, 부활사유, 심사결과, 미납보험료, 부활처리일시)가 자사 DB에 저장되고 계약상태가 '유효'로 갱신된다.
Basic Path	
1.	계약관리담당자는 부활 신청 정보 (피보험자 이름, 주민등록번호, 증권번호)를 입력한다
2.	시스템은 해당 증권번호로 계약정보 (계약상태, 실효일, 보험기간, 보험가입 금액, 보험료, 미납금액, 미납회차)를 조회하여 화면에 출력한다 (A1)
3.	계약관리담당자는 부활신청 사유(부활사유, 미납보험료 납입확인, 건강상태 변동여부(있음/없음), 최종납입일, 부활희망일)를 입력하고 부활신청 버튼을 누른다
4.	시스템은 부활 신청 내용(증권번호, 피보험자정보, 부활사유, 미납보험료, 부활희망일, 신청일시)과 심사요청 버튼을 화면에 출력한다
5.	계약관리담당자는 심사요청 버튼을 누른다 <<include>> [심사를 요청한다] 시나리오를 수행
6.	시스템은 심사 결과(승인/거절)를 화면에 출력한다
7.	계약관리담당자는 확인 버튼을 누른다
8.	시스템은 부활 처리 결과(증권번호, 부활사유, 심사결과, 미납보험료, 부활처리일시)를 자사 DB에 저장하고 계약상태를 유효로 갱신한다 (E1)
Alternate	
A1. 실효된 계약이 아닌 경우	
2.	시스템은 "부활 신청이 불가능한 계약입니다" 메시지와 계약상태(유지/해지/만기)를 화면에 출력한다.
3.	계약관리담당자는 확인 버튼을 누른다.

➤ 시스템이 종료된다.	
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	계약관리담당자는 다시 시도 버튼을 누른다.
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

심사를 요청한다	
액터	언더라이터, 한국신용정보원(ICIS)
사전조건	✓ 유스케이스 시나리오 [배서를 관리한다] Basic Path 5번 또는 [부활을 관리한다] Basic Path 5번에서 시작되었다.
사후조건	➤ 심사 요청 내역(심사유형, 심사결과, 할증조건 또는 거절사유, 심사자 정보, 심사 완료 일시)이 자사 DB에 저장되고 요청상태가 '심사 완료'로 갱신된다.
Basic Path	
1.	계약관리담당자는 심사 요청 정보 (피보험자 이름, 주민등록번호, 증권번호, 심사유형(배서/부활))를 입력한다.
2.	시스템은 해당 증권번호로 계약정보(계약상태, 보험기간, 보험가입금액, 보험료, 미납여부)를 조회하여 화면에 출력한다.
3.	계약관리담당자는 심사 요청 사유(- 배서인 경우: 배서유형(보험가입금액변경, 납입주기변경, 특약추가, 특약삭제, 수익자변경), 변경 전 내용, 변경 후 내용, 변경사유 - 부활인 경우: 부활사유, 미납보험료 납입확인, 건강상태 변동여부(있음/없음), 최종 납입일, 부활 희망일)를 입력하고 심사요청 버튼을 누른다. (A1)
4.	시스템은 심사 요청 내용(피보험자 정보, 증권번호, 심사유형, 요청사유, 요청일시)을 언더라이터에게 전달하고 요청상태를 심사대기로 갱신하여 화면에 출력한다.
5.	계약관리담당자는 확인 버튼을 누른다.
6.	시스템은 언더라이터에게 온 심사결과(승인/할증/거절)을 화면에 출력한다.
7.	계약관리담당자는 확인 버튼을 누른다.

8.	시스템은 요청상태를 심사완료로 갱신 후, 심사 요청 내역 (심사유형, 심사 결과, 할증조건 또는 거절사유, 심사자 정보, 심사 완료 일시)을 자사 DB에 저장한다. (E1)
Alternate	
A1. 추가 서류가 필요한 경우	
2.	시스템은 "추가 서류가 필요합니다" 메시지와 필요한 서류 목록 (진단서, 사고확인서, 신분증 사본)을 화면에 출력한다.
3.	계약관리담당자는 필요한 서류를 시스템에 첨부한다. (Basic Path 3번으로 복귀한다)
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	계약관리담당자는 다시 시도 버튼을 누른다.
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

제지급금을 관리한다	
액터	계약관리담당자
사전조건	✓ 피보험자의 계약 정보가 시스템에 등록되어 있어야 한다. ✓ 계약이 지급 가능한 상태(유효/만기)이어야 한다.
사후조건	➤ 지급 처리 결과(증권번호, 지급유형, 지급금액, 지급일시, 처리담당자, 승인일시)가 자사 DB에 저장되고 계약정보가 갱신된다.
Basic Path	
1.	계약관리담당자는 지급 요청 정보(피보험자 이름, 주민등록번호, 증권번호, 지급유형(만기환급금/중도인출금/해지환급금/배당금))를 입력한다.
2.	시스템은 해당 증권번호로 계약정보(계약상태, 보험기간, 보험가입금액, 보험료, 납입회차, 계약자 계좌정보)를 조회하여 화면에 출력한다. (A1)
3.	계약관리담당자는 지급요청 확인 버튼을 누른다.
4.	시스템은 지급금을 자동 산출하여 지급금 정보(지급유형, 산출금액, 산출기준, 공제항목, 최종지급금액)를 화면에 출력한다.
5.	계약관리담당자는 승인 버튼을 누른다. (A2)

6.	시스템은 계약자 계좌로 지급금(증권번호, 지급유형, 지급금액, 지급계좌)을 자동으로 이체 처리한다.
7.	시스템은 계약자에게 지급안내장(증권번호, 지급유형, 지급금액, 지급일시, 입금계좌)을 자동으로 발송한다.
8.	계약관리담당자는 확인 버튼을 누른다.
9.	시스템은 지급 처리 결과(증권번호, 지급유형, 지급금액, 지급일시, 처리담당자, 승인일시)를 자사 DB에 저장하고 계약정보를 갱신한다. (E1)
Alternate	
A1. 계약이 지급 불가능한 상태인 경우	
2.	시스템은 "지급이 불가능한 계약입니다" 메시지와 계약상태를 화면에 출력한다.
3.	계약관리담당자는 확인 버튼을 누른다.
➤ 시스템이 종료된다.	
A2. 지급금 산출 결과를 재확인해야 하는 경우	
2.	계약관리담당자는 반려 버튼을 누른다.
3.	시스템은 반려 사유 입력 화면을 출력한다.
4.	계약관리담당자는 반려 사유를 입력하고 확인 버튼을 누른다.
5.	시스템은 반려 결과(증권번호, 반려사유, 반려일시)를 자사 DB에 저장한다.
➤ 시스템이 종료된다.	
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	계약관리담당자는 다시 시도 버튼을 누른다.
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

분납/수금을 관리한다	
액터	계약관리담당자
사전조건	✓ 납입기일이 도래한 수금대상 계약이 존재하여야 한다. 수금대상 ✓ 계약의 자동이체 계좌정보가 시스템에 등록되어 있어야 한다.

사후조건	➤ 수금 처리 결과(증권번호, 피보험자 이름, 수금금액, 수금일시, 납입 회차, 처리결과)가 자사 DB에 저장되고 계약정보가 갱신된다.
Basic Path	
1.	계약관리담당자는 수금 실행 버튼을 누른다.
2.	시스템은 납입기일이 도래한 수금대상 계약 목록(증권번호, 피보험자 이름, 보험료, 납입기일, 자동이체 계좌정보, 납입회차)을 추출하여 화면에 출력한다.
3.	계약관리담당자는 일괄처리 버튼을 누른다.
4.	시스템은 수금대상 계약의 자동이체를 일괄 실행하고 처리결과(성공건수, 실패건수, 총수금금액)를 화면에 출력한다. (A1)
5.	계약관리담당자는 확인 버튼을 누른다.
6.	시스템은 수금 처리 결과(증권번호, 피보험자 이름, 수금금액, 수금일시, 납입회차, 처리결과)를 자사 DB에 저장하고 계약정보를 갱신한다.
Alternate	
A1. 자동이체 실패한 계약이 존재하는 경우	
2.	시스템은 자동이체 실패 계약자에게 미납안내(증권번호, 피보험자 이름, 미납금액, 납입기한, 납입방법)를 자동으로 발송하고 계약상태를 미납으로 갱신한다.
3.	시스템은 미납 처리 결과 (증권번호, 미납금액, 미납회차, 안내발송일시)를 자사 DB에 저장한다. (Basic Path 4번으로 복귀한다)
A1-1. 미납이 지속되어 이관이 필요한 경우	
2.	시스템은 이관대상 계약 목록 (증권번호, 피보험자 이름, 미납금액, 미납회차, 미납기간)을 계약관리담당자 화면에 출력한다.
3.	계약관리담당자는 이관유형 (방문수금/해지처리)을 선택하고 이관처리 버튼을 누른다.
4.	시스템은 이관처리 결과 (증권번호, 이관유형, 이관일시, 담당자 정보)를 자사 DB에 저장하고 계약상태를 갱신한다.
5.	계약관리담당자는 확인 버튼을 누른다.
➤ 시스템이 종료된다.	
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	계약관리담당자는 다시 시도 버튼을 누른다.
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를

	통보한다.
➤ 시스템이 종료된다.	

만기계약을 관리한다	
액터	계약관리담당자
사전조건	✓ 만기일이 도래한 계약이 시스템에 존재하여야 한다. ✓ 만기대상 계약자의 연락처 정보가 시스템에 등록되어 있어야 한다.
사후조건	➤ 재계약 의사 확인 결과(증권번호, 재계약의사, 확인일시)가 자사 DB에 저장되고 계약상태가 갱신된다.
Basic Path	
Ps) 액터의 개입 없이 시스템이 순차적으로 내용을 추출하고 반복하는 과정이기에 시스템이 내부 시나리오에서 연속으로 등장한다	
1.	시스템은 만기일이 도래한 계약 목록(증권번호, 피보험자 이름, 보험기간, 만기일, 보험가입금액, 만기환급금)을 자동으로 추출한다.
2.	시스템은 만기대상 계약자에게 만기안내장(증권번호, 피보험자 이름, 만기일, 만기환급금, 재계약안내)을 자동으로 발송한다.
3.	시스템은 만기안내 발송 결과(증권번호, 발송일시, 발송방법)를 자사 DB에 저장한다.
4.	계약관리담당자는 재계약 의사 확인 대상 목록(증권번호, 피보험자 이름, 만기일, 안내발송일시)을 조회한다.
5.	계약관리담당자는 재계약 의사 확인 결과(있음/없음)를 입력한다. (A1)
6.	시스템은 재계약 의사 확인 결과(증권번호, 재계약의사, 확인일시)를 자사 DB에 저장하고 계약상태를 만기로 갱신한다. (E1)
Alternate	
A1. 고객으로부터 재계약 의사 회신이 없는 경우	
2.	시스템은 회신 기한이 초과된 미 회신 계약 목록(증권번호, 피보험자 이름, 만기일, 안내발송일시)을 계약관리담당자 화면에 출력한다.
3.	계약관리담당자는 만기 처리 대상을 선택한다.
4.	시스템은 만기 처리 결과(증권번호, 만기일, 만기환급금, 처리일시)를 자사 DB에 저장하고 계약상태를 만기종료로 갱신한다.
➤ 시스템이 종료된다.	
A2. 고객이 재계약 의사가 없는 경우	

2.	시스템은 만기 처리 안내(증권번호, 피보험자 이름, 만기환급금, 지급예정일)를 화면에 출력한다.
3.	계약관리담당자는 확인 버튼을 누른다.
4.	시스템은 만기 처리 결과(증권번호, 만기일, 만기환급금, 처리일시)를 자사 DB에 저장하고 계약상태를 만기종료로 갱신한다.
➤ 시스템이 종료된다.	
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	계약관리담당자는 다시 시도 버튼을 누른다.
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

상품을 설계한다	
액터	상품개발자, 금융감독원
사전조건	✓ 상품 기획이 완료된 상태이다.
사후조건	➤ 상품 상태가 '설계 완료'로 변경된다.
Basic Path	
1.	상품개발자는 '신규 상품 설계' 버튼을 누른다.
2.	시스템은 상품 기본정보 입력 화면 (상품명, 상품코드, 상품유형, 적용대상, 판매채널, 보험기간, 납입기간)을 출력한다.
3.	상품개발자는 상품 기본정보 (상품명, 상품코드, 상품유형, 적용대상, 판매채널, 보험기간, 납입기간)를 입력한다.
4.	시스템은 담보정보 입력 화면 (주담보명, 보장내용, 보험가입금액 한도, 면책조건, 지급조건)을 출력한다.
5.	상품개발자는 담보정보 (주담보명, 보장내용, 보험가입금액 한도, 면책조건, 지급조건)를 입력한다.
6.	시스템은 가입조건 입력 화면 (가입가능연령, 가입제한조건, 직업조건, 건강조건, 계약제한사유)을 출력한다.
7.	상품개발자는 가입조건 (가입가능연령, 가입제한조건, 직업조건, 건강조건, 계약제한사유)를 입력한다.

8.	시스템은 보험요율 입력 및 산정 화면 (기초요율, 위험률, 예정이율, 사업비율, 할인/할증요율)을 출력한다.
9.	상품개발자는 요율 정보(기초요율, 위험률, 예정이율, 사업비율, 할인/할증요율)를 입력하고 '요율 산정' 버튼을 누른다
10.	시스템은 보험요율 및 예상보험료 (기초요율, 적용요율, 예상보험료, 손익예상치)를 출력한다. (A1)
11.	상품개발자는 '특약정보 입력' 버튼을 누른다. (A2)
12.	시스템은 특약정보 입력 화면 (특약명, 보장내용, 가입조건, 특약보험료, 중복가입 가능여부)을 출력한다.
13.	상품개발자는 특약정보(특약명, 보장내용, 가입조건, 특약보험료, 중복가입 가능여부)를 입력한다.
14.	시스템은 입력된 상품정보 요약 화면 (상품 기본정보, 담보정보, 가입조건, 보험요율, 특약정보)을 출력한다.
15.	상품개발자는 '저장' 버튼을 누른다.
16.	시스템은 상품 설계 정보를 자사 DB에 저장하고 상품 상태를 설계완료로 변경한 후 설계 완료 화면(상품명, 상품코드, 설계완료일시, 다음단계 안내)을 출력한다. (E1)
17.	<<extend>> [상품 인가를 요청한다] 시나리오를 진행한다 (A3)
Alternate	
A1. 보험요율 재산정이 필요한 경우	
1.	상품개발자는 요율 요소 (기초요율, 위험률, 예정이율, 사업비율, 할인/할증요율)를 수정 입력한다.
2.	시스템은 수정된 요율 요소를 반영하여 보험요율 및 예상보험료를 다시 계산하여 화면에 출력한다.
3.	상품개발자는 재산정 결과 확인 버튼을 누른다. (Basic Path 10번으로 복귀한다.)
A2. 특약이 없는 상품인 경우	
1.	상품개발자는 특약 없음 체크박스를 선택한다.
2.	시스템은 특약정보 입력 단계를 생략하고 상품정보 요약 화면을 출력한다. (Basic Path 15번으로 복귀한다.)
A2. 인가가 필요 없는 경우	
2.	시스템이 메시지 ("인가가 필요 없는 상품입니다.") 를 출력한다.
3.	상품 개발자는 확인 버튼을 누른다.
4.	시스템을 종료한다.
Exception	

E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 저장 실패 화면(실패 사유, 재시도 안내)을 출력한다.
3.	상품개발자는 재시도를 요청하거나 (Basic Path 15로 복귀) '임시저장 후 종료'를 선택한다.
4.	시스템은 임시저장 완료 화면(상품명, 상품코드, 임시저장일시)을 출력한 후 종료한다.
➤ 시스템이 종료된다.	

상품 인가를 요청한다	
액터	상품개발자, 금융감독원
사전조건	✓ 상품 설계가 완료된 상태이다.
사후조건	➤ 상품 인가의 결과가 결정된다.
Basic Path	
1.	상품개발자는 상품 목록에서 인가 요청 대상 상품을 선택한다.
2.	시스템은 선택된 상품의 인가 요청 화면(상품명, 상품코드, 상품유형, 보장내용, 가입조건, 보험요율, 특약정보, 첨부서류 목록)을 출력한다
3.	상품개발자는 해당 상품의 '인가 요청' 버튼을 누른다.
4.	시스템은 인가 요청서 입력 화면(요청일자, 요청사유, 제출기관명, 첨부서류 항목)을 출력한다.
5.	상품개발자는 인가 요청 정보(요청일자, 요청사유, 제출기관명)와 첨부서류(상품설명서, 약관, 요율서, 상품개발 근거자료)를 입력한다.
6.	시스템은 인가 요청 데이터를 생성, 제출 상태를 '인가요청'으로 변경한 후 금융감독원에 인가 요청 정보를 전송한다. (A1) (E1)
7.	시스템은 인가 결과(승인여부, 승인일자, 보완요청사항)를 화면에 출력한다. (A2)
8.	상품개발자는 '결과 확인' 버튼을 누른다
9.	시스템이 인가 결과에 따라 상품 상태(인가 완료, 인가 불허, 보완 요청)를 변경한다
Alternate	
A1. 인가 요청을 취소하는 경우	
1.	상품개발자는 '인가 요청 취소' 버튼을 누른다.
2.	시스템은 취소 확인용 메시지를 출력한 후 인가 요청 상태를 '취소'로 변경

	한다.
➤ 시스템이 종료된다.	
A2. 보완 요청이 발생할 경우	
2.	시스템은 보완 요청 사항(약관 수정사항, 요율 수정사항, 첨부서류 보완사항)을 출력한다.
3.	상품개발자는 보완이 필요한 항목을 수정하고 '재제출' 버튼을 누른다. (Basic Path 6번으로 복귀한다.)
Exception	
E1. 금융감독원 API가 응답하지 않는 경우	
2.	시스템은 "금융감독원 API가 응답하지 않아 정보를 가져오는 것을 실패했습니다." 를 출력한다.
3.	상품개발자는 '임시저장' 버튼을 누른다.
4.	시스템은 현재 상태를 자사 DB에 임시저장 후 시스템을 종료한다.
➤ 시스템이 종료된다.	

사고를 접수한다	
액터	보험가입자, 보험사 직원
사전조건	✓ 보험가입자가 자사와 유효한 보험 계약을 체결한 상태이다.
사후조건	➤ 사고가 접수되어 현장출동을 한다.
Basic Path	
1.	보험가입자는 '사고 접수' 버튼을 누른다.
2.	시스템은 사고 접수 화면(보험 증권번호, 사고 일시, 사고 경위, 피해 내용)을 출력한다.
3.	보험가입자는 서류를 등록 및 정보를 입력 후 '접수'버튼을 누른다
4.	시스템은 계약 정보 (피보험자명, 보험 종류, 보장 범위)를 화면에 출력한다.
5.	보험가입자는 사고 관련 서류(사고경위서, 진단서, 청구서류)를 업로드하고 제출 버튼을 누른다. (A1)
6.	시스템은 접수 내용(보험 증권번호, 사고 일시, 사고 경위, 피해 내용, 사고 경위서, 진단서, 청구서류)을 자사 DB에 저장, '사고 접수 번호'와 '정상적으로 접수되었습니다' 문구를 출력한 후 사건의 상태를 '현장 조사 필요'로 변경한다. (E1)
Alternate	

A1. 보험가입자가 서류를 즉시 제출할 여건이 안 되는 경우	
1.	보험가입자는 '서류 나중에 제출' 버튼을 누른다.
2.	시스템은 접수 내용을 '서류 미제출' 상태로 DB에 저장한 후 사고 접수 번호와 서류 제출 기한을 화면에 출력한다. (Basic Path 6번으로 복귀한다.)
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	보험가입자는 '다시 시도' 버튼을 누른다.
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

손해조사를 한다	
액터	손해사정인, SIU(보험사고조사팀)
사전조건	✓ [사고를 접수한다] 시나리오가 진행된 상태이다. ✓ 보험사 직원에 의해 자사 손해사정인에게 사건이 배정된 상태이다. ✓ 현장 출동을 통해 자사 직원들이 사고 증거자료를 시스템에 업로드한 상태이다.
사후조건	➤ 사고 접수 상태가 '결재 필요'로 변경된다.
Basic Path	
1.	손해사정인은 '사고 접수' 목록 화면에서 사고 접수 번호를 입력한다.
2.	시스템은 접수 내용(보험 증권번호, 사고 일시, 사고 경위, 피해 내용, 사고 경위서, 진단서, 청구서류)을 화면에 출력한다.
3.	손해사정인은 '현장조사 자료' 버튼을 누른다. (A1)
4.	시스템은 사고 관련 자료 (사고현장 사진, 블랙박스 영상, 수리 견적)이 정리된 문서를 화면에 출력한다. (A2)
5.	손해사정인은 피해 규모를 산정한 후 손해액 (치료비 실비, 휴업손해, 위자료, 수리비, 과실 비율)을 시스템에 입력한다. (A3)
6.	시스템은 입력된 정보들을 바탕으로 지급품의서 초안을 작성해 화면에 출력한다.
7.	손해사정인은 지급품의서 초안을 바탕으로 소견(손해액 적정성 판단, 과실비

	을 의견, 특이사항)을 시스템에 작성한다.
8.	시스템은 최종 지급품의서를 화면에 출력한다.
9.	손해사정인은 '결재' 버튼을 누른 후 사원번호를 입력한다.
10.	시스템은 지급품의서를 자사 DB에 저장 후 사고 접수 상태를 '결재 필요'로 변경한다. (E1)
11.	<<extend>> [보험금을 지급한다] 시나리오를 진행한다. (A4)
Alternate	
A1. 사고 접수된 내용이 보험계약과 관련이 없는 경우	
1.	손해사정인은 '보험 처리 반려' 버튼을 누른다.
2.	시스템은 '사원번호', '반려 사유' 입력창을 출력한다.
3.	손해사정인은 사원번호와 반려 사유를 입력한다.
4.	시스템은 문서 형식으로 전환한 후 자사 DB에 저장, 사고 접수 상태를 '반려'로 변경한다.
➤ 시스템이 종료된다.	
A2. 보험사기가 의심되는 경우	
1.	손해사정인은 '보험 사기 평가' 버튼을 누른다.
2.	시스템은 '보험 사기로 판단되어 조사를 요청합니다.' 문구를 출력한 후 사용자가 '실시한다'를 입력하도록 유도한다.
3.	손해사정인은 '실시한다'를 입력한다.
4.	시스템은 사고 접수 상태를 '보험 사기 조사'로 변경한다.
➤ 사건을 SIU (보험사기조사팀)에 위임하고 시스템을 종료한다.	
A3. 피해 규모가 자체 조사 범위를 초과할 경우	
1.	손해사정인은 '손해조사를 위탁한다' 버튼을 누른다.
2.	<<extend>> '손해조사를 위탁한다' 시나리오를 진행한다.
3.	위탁 조사 결과가 시스템에 반영된 후 Basic Path 5번으로 복귀한다.
A4. 지급 품의서가 반려된 경우	
1.	Basic Path 5번으로 복귀해 다시 진행한다
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	손해사정인은 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 10번으로 복귀한다.)

4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

손해조사를 위탁한다	
액터	손해사정인, 협력업체
사전조건	✓ 유스케이스 시나리오 [손해조사를 한다]의 A3 2번에서 시작되었다. ✓ 피해 규모가 자사 조사 범위를 초과할 것으로 판단되어 위탁하기로 결정했다.
사후조건	➤ [손해조사를 한다]의 A3-3번으로 복귀한다.
Basic Path	
2.	시스템은 협력업체로 등록된 업체들의 리스트를 화면에 출력한다. (A1)
3.	손해사정인은 사건에 맞는 협력업체를 클릭한다.
4.	시스템은 보험계약 내용, 청구서류, 진단서, 사고경위서, 사고 관련 자료 (사고현장 사진, 블랙박스 영상, 수리 견적) 중 어떤 것을 보낼 것인지 체크리스트 화면을 제시한다.
5.	손해사정인은 필요한 자료를 선택 후 '다음'을 클릭한다.
6.	시스템은 손해사정 위탁 의뢰서를 문서로 정리해 화면에 출력한다.
7.	손해사정인은 '제출한다'를 클릭한다.
8.	시스템은 협력업체에게 문서 전달 후 자사 DB에 저장, 사고 조사 상태를 '손해조사 위탁'으로 변경한다. (E1)
9.	시스템은 손해조사 위탁 결과가 도착하면 알림을 띄운다. ([손해조사를 한다]의 A3-3번으로 복귀한다.)
Alternate	
A1. 기존에 등록된 협력업체가 아닌 새로운 기관일 경우	
✓ 자사 회의와 계약을 통해 새로운 업체와 협력하기로 한 상태이다.	
1.	손해사정인은 '신규 협력업체 등록 요청' 버튼을 누른다.
2.	시스템은 협력업체 등록 문서 작성란(업체명, 업체유형, 담당업무, 연락처, 계약조건, 평가등급)을 화면에 출력한다.
3.	손해사정인은 내용 입력 후 '반영' 버튼을 누른다.
4.	시스템은 협력업체 관리 부서에 업데이트를 요청한다. (업데이트 완료 시 Basic Path 2번으로 복귀한다.)

Exception	
E1. 협력업체의 시스템이 동작하지 않는 경우	
2.	시스템은 “손해조사 위탁 요청에 실패했습니다”를 출력한 후 현재 문서를 자사 DB에 저장, 사고조사 상태를 ‘임시저장’으로 변경한다
➤ 시스템을 종료한 후 협력업체 시스템의 복구를 기다린다.	

보험금을 지급한다	
액터	보상담당자, 피보험자
사전조건	✓ 유스케이스 시나리오 [손해조사를 한다]의 Basic Path 11번에서 시작되었다. ✓ 자사 절차에 의해 지급품의서가 승인된 상태이다. ✓ 피보험자에게 보험금이 지급될 예정이니 확인 후 바로 응답해달라고 한 상태이다.
사후조건	➤ 보험 처리가 정상적으로 종결된다.
Basic Path	
1.	보상담당자는 '결제 완료' 상태의 사건을 선택한다.
2.	시스템은 품의서를 기준으로 항목 (치료비 실비, 휴업손해, 위자료, 수리비) 별 최종 결정보험금, 지급에 따른 자사 보유금 예상치를 화면에 문서로 출력한다.
3.	보상담당자는 '다음' 버튼을 누른다.
4.	시스템은 보험 계약에 따른 수익자의 정보 (계좌번호, 은행명, 성명, 연락처)를 화면에 출력한다.
5.	보상담당자는 시스템에 사원번호를 입력 후 '최종 이체 및 종결' 버튼을 누른다.
6.	시스템은 자사 보유액에서 최종 결정보험금을 차감한 후 계좌이체 처리한다. (E1)
7.	보상담당자는 '다음' 버튼을 누른다.
8.	시스템은 보험금 산출 결과, 사유, 수익자 계좌번호, 은행, 성명, 연락처, 지급 내역 (날짜, 시간)을 문서로 작성 후 자사 DB에 저장, 사고 접수자에게 알림을 보낸 후 사건의 상태를 '지급 완료'로 변경한다. (E2)
9.	보상담당자는 '확인' 버튼을 누른 후 피보험자의 응답을 기다린다.
10.	시스템은 피보험자가 수령 확인을 응답하면 화면에 알림으로 출력한다. (A1)

11.	보상담당자는 알림을 확인했음을 시스템에 표시한 후 '사건 종결' 버튼을 누른다. (A2)
12.	시스템은 사건의 상태를 '종결'로 변경한다.
Alternate	
A1. 피보험자가 지급 금액에 이의를 제기한 경우	
1.	<<extend>> [이의 제기를 처리한다] 시나리오를 진행한다. (Basic Path 10번으로 복귀한다.)
A2. 제3자 과실이 존재하여 구상을 청구해야 하는 경우	
1.	보상담당자는 '구상 대기 등록' 버튼을 누른다.
2.	시스템은 사건의 상태를 '지급 완료/구상 처리 필요'로 변경한다.
➤ 시나리오를 종료한다.	
Exception	
E1. 은행 시스템 오류로 인해 이체가 실패한 경우	
2.	시스템은 "이체 실패" 오류 메시지를 화면에 출력한다.
3.	보상담당자는 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 6번으로 복귀한다.)
4.	시스템이 재시도 후에도 이체가 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	
E2. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	보상담당자는 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 8번으로 복귀한다.)
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

구상을 처리한다	
액터	보상담당자, 법률과
사전조건	✓ 보험금이 정상적으로 피보험자에게 지급이 완료된 상태이다. ✓ 해당 사고에 피보험자가 아닌 제3자의 과실이 존재하는 상태이다.

	✓ 사건의 상태가 '지급 완료/구상 처리 필요'인 상태이다.
사후조건	➤ 보험 처리가 정상적으로 종결된다.
Basic Path	
1.	보상담당자는 '지급 완료/구상 처리 필요' 상태의 사건을 클릭한다.
2.	시스템은 보험금 산출 결과, 사유, 지급 내역 (날짜, 시간)을 화면에 출력한다.
3.	보상담당자는 사고 정보, 지급 내역, 제3자가 지급해야 할 과실 비율을 시스템에 입력한다.
4.	시스템은 제3자(가해자)가 지불해야 할 금액을 계산, 문서(일시, 장소, 경위, 피보험자 및 가해자 정보, 과실비율, 보험금 지급 내역, 구상 청구 금액, 사고경위서, 진단서, 수리견적서, 보험계약 조항, 약관 근거, 납부기한 및 입금 계좌)로 생성해 화면에 출력한다.
5.	보상담당자는 가해자 보험사 연락처를 입력한 후 '구상 청구서 발송' 버튼을 누른다. (A1)
6.	시스템은 구상 청구서를 자사 DB에 저장한 후 가해자에게 구상 청구서를 발송한다. (E1)
7.	시스템은 입금이 완료되면 사건을 '종결' 상태로 변경한다. (A2)
Alternate	
A1. 가해자가 위임한 보험사가 없는 경우	
1.	보상담당자는 대신 가해자 연락처를 입력한 후 '구상 청구서 발송' 버튼을 누른다. (Basic Path 6번으로 복귀한다.)
A2. 제3자가 구상 청구서를 수용하지 않는 경우	
1.	<<extend>> [이의 제기를 처리한다] 시나리오를 진행한다. (Basic Path 4번으로 복귀한다.)
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	보상담당자는 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 7번으로 복귀한다.)
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

이의 제기를 처리한다	
액터	보상담당자, 법률과
사전조건	✓ 유스케이스 시나리오 [보험금을 지급한다] 또는 [구상을 처리한다] 시나리오 진행 중 이의 제기가 접수된 상태이다.
사후조건	➤ 호출한 시나리오로 복귀한다.
Basic Path	
2.	시스템은 이의 제기 내용(이의 사유, 제기자 정보, 원 지급/청구 내역)을 화면에 출력한다.
3.	보상담당자는 기존 산출 근거와 비교하여 재검토 의견(수용 여부, 조정 금액, 사유)을 시스템에 작성한다. (A1)
4.	시스템은 기각 사유서를 문서로 작성하여 화면에 출력한다.
5.	보상담당자는 '발송' 버튼을 누른다.
6.	시스템은 기각 사유서(이의 사유, 제기자 정보, 원 지급/청구 내용, 수용 여부(기각), 조정 금액, 사유)를 자사 DB에 저장한 후 이의 제기자에게 발송한다. (A2) (E1)
7.	호출한 시나리오로 복귀한다.
Alternate	
A1. 이의 제기가 타당한 경우	
1.	보상담당자는 이의 제기 수용 버튼을 누른다.
2.	시스템은 빈 손해조사 요청서를 화면에 출력한다.
3.	보상담당자는 근거와 재검토 의견 (수용 여부, 조정 금액, 사유)를 시스템에 작성한다.
4.	시스템은 수용 여부, 조정 금액, 사유를 자사 DB에 저장한 후 사건의 상태를 '재조사 필요'로 변경한다
➤ 시스템을 종료하고 [손해조사를 한다]를 다시 진행한다.	
A2. 이의 제기자가 결과에 불복해 소송 과정을 거쳐야 하는 경우	
1.	보상담당자는 '법률과 이관' 버튼을 누른다.
2.	시스템은 이관 사유 입력창을 화면에 출력한다.
3.	보상담당자는 이관 사유를 입력한 후 '제출' 버튼을 누른다.
4.	시스템은 사건 관련 서류(보험금 산출 근거, 이의 제기 내용)와 이관 사유를 법률과에 전달하고, 사건 상태를 '법률과 이관'으로 변경한다.

5.	시스템은 법률과 처리가 완료되면 처리 결과를 보상담당자에게 알림으로 보낸다. (Basic Path 3번으로 복귀한다.)
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 "저장 실패" 오류 메시지를 화면에 출력한다.
3.	보상담당자는 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 6번으로 복귀한다.)
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

보상 평가를 관리한다	
액터	보상담당자
사전조건	✓ [손해조사를 한다] 시나리오를 완료한 상태이다.
사후조건	➤ 보상평가자료가 생성된다.
Basic Path	
1.	보상담당자는 보상평가 관리 메뉴를 선택한다.
2.	시스템은 보상평가 조회 조건 입력 화면(조회기간, 보험종목, 보상유형)을 출력한다.
3.	보상담당자는 조회 조건(조회기간, 보험종목, 보상유형)을 입력하고 조회를 요청한다.
4.	시스템은 기간별 보상통계 현황(접수건수, 처리건수, 미결건수, 지급보험금 합계, 평균처리일수)을 화면에 출력한다.
5.	보상담당자는 '손해액 분석' 버튼을 누른다.
6.	시스템은 손해액 분석 결과(보험종목별 손해액, 손해율, 전월 대비 증감, 손해액 추이)를 화면에 출력한다 (A1)
7.	보험 담당자는 '마감 통계 산출' 버튼을 누른다. (E1)
8.	시스템은 마감통계 데이터(월별 접수현황, 처리현황, 지급현황, 손해율)를 집계하고 마감통계 결과 화면을 출력한다.
9.	보상담당자는 '결과 확인' 버튼을 누른다.

10.	시스템은 "보상평가자료 생성 완료" 메시지를 출력한다.
Alternate	
A1. 특정 보험 종목만 분석하는 경우	
1.	보상담당자는 특정 보험종목을 선택한다.
2.	시스템은 선택된 보험종목 기준으로 손해액 분석 결과를 출력한다. (Basic Path 6번으로 복귀한다)
Exception	
E1. 데이터 집계 중 오류 발생 시	
2.	시스템은 "데이터 집계 실패" 메시지를 출력한다
3.	보상담당자는 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 7번으로 복귀한다.)
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

협력업체를 관리한다	
액터	보상담당자
사후조건	➤ 협력업체 정보 및 계약/평가 정보가 시스템에 저장된다.
Basic Path	
1.	보상담당자는 협력업체 관리 메뉴를 선택한다.
2.	시스템은 협력업체 조회 조건 입력 화면 (업체명, 업체유형, 계약상태, 평가 등급)을 출력한다.
3.	보상담당자는 조회 조건 (업체명, 업체유형, 계약상태, 평가등급)를 입력하고 조회를 요청한다.
4.	시스템은 협력업체 목록 (업체명, 업체유형, 담당업무, 계약상태, 평가등급) 을 화면에 출력한다.
5.	보상담당자는 협력 업체를 선택한다. (A1)
6.	시스템은 협력업체 정보 입력 화면 (업체명, 업체유형, 담당업무, 연락처, 계 약조건, 평가등급)을 출력한다.
7.	보상담당자는 협력업체 정보 (업체명, 업체유형, 담당업무, 연락처, 계약조 건, 평가등급)를 입력 또는 수정한다.
8.	보상담당자는 저장 버튼을 누른다. (E1)

9.	시스템은 협력업체 정보 및 변경 이력을 DB에 저장 후 "저장을 완료했습니다" 메시지를 출력한다.
Alternate	
A1. 신규 협력 업체를 등록할 경우	
1.	보상담당자는 '신규 협력업체 등록' 버튼을 누른다.
2.	시스템은 협력업체 신규 등록 화면(업체명, 업체유형, 담당업무, 연락처, 계약조건, 평가등급)을 출력한다.
3.	보상담당자는 신규 협력업체 정보를 입력 후 저장 버튼을 누른다.
4.	시스템은 신규 협력업체 정보와 등록 이력을 자사 DB에 저장 후 "저장을 완료했습니다" 메시지를 출력한다. (Basic Path 5번으로 복귀한다.)
Exception	
E1. DB 서버 오류로 저장 불가능한 경우	
2.	시스템은 저장 실패 메시지를 출력한다.
3.	보상담당자는 '다시 시도' 버튼을 누른다. (성공 시 Basic Path 8번으로 복귀한다.)
4.	시스템이 재시도 후에도 저장 불가능한 경우 시스템은 관리자에게 오류를 통보한다.
➤ 시스템이 종료된다.	

IV. Class Diagram

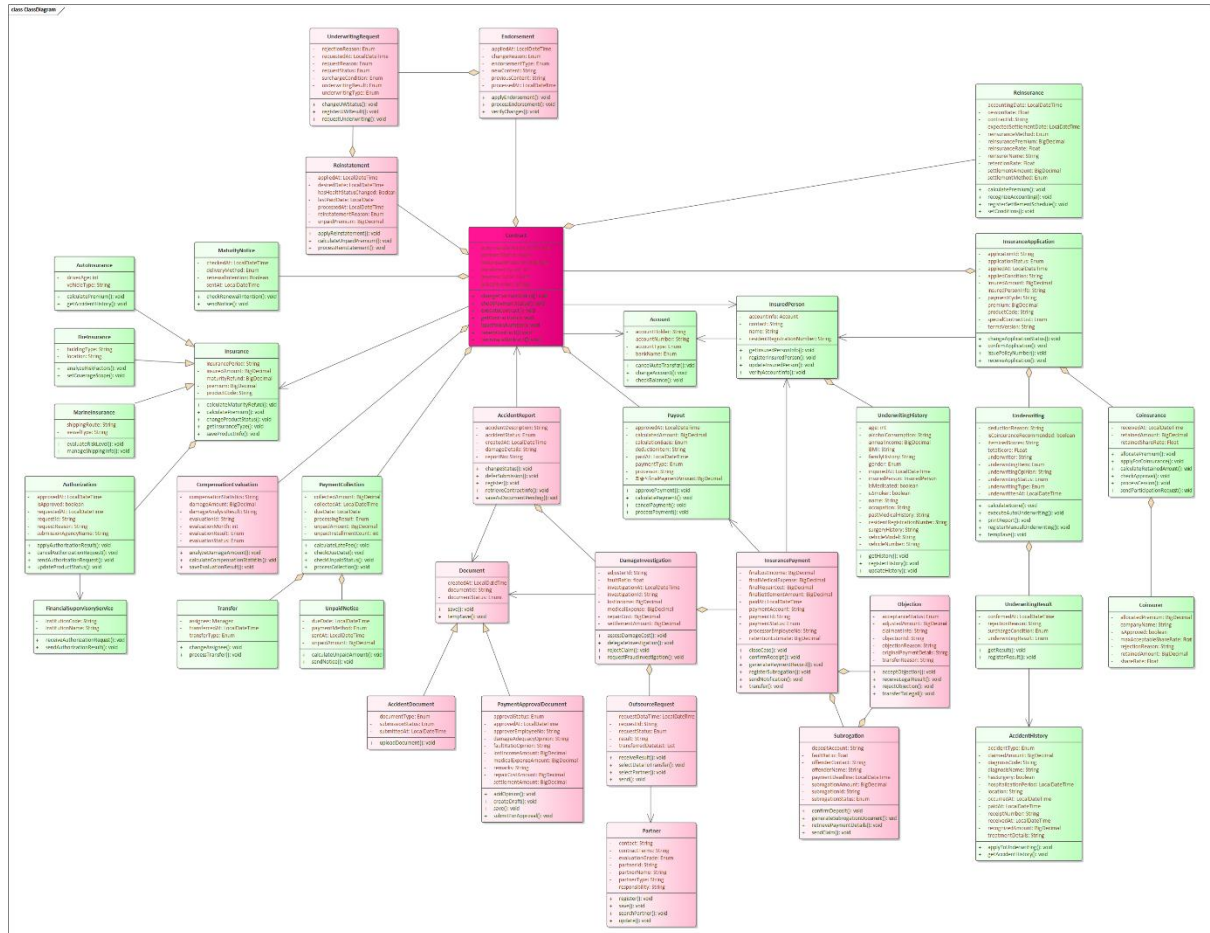


그림 2: Class Diagram(클래스 다이어그램)

구성 사유

이전 유스케이스 다이어그램에서 뽑은 유스케이스들을 실제 기능으로 보고, 실제 구현을 하자고 팀원들과 결정하였다. 그렇기 때문에 유스케이스들이 클래스 다이어그램에서 핵심적인 기능들로 뽑히게 되었다. 유스케이스는 동사를 기준으로 유스케이스를 선정한 것과 달리 클래스 다이어그램의 경우 명사를 가지고 선택을 해야 했다. 그렇기 때문에 어떠한 관계가 나타나는지 표현하기 위해 클래스 내부의 상태와 함수들, 그리고 클래스 간의 관계를 적절하게 표현할 필요성이 있었다. 이를 잘 표현하기 위해 이번 팀프로젝트에서 회의를 반복하며 수정을 해 나갔다.

클래스 선택 사유

우리는 먼저 PDF에서 제시된 To-Be 프로세스 체계도를 기준으로 삼았다. 체계도의 세부 영역들을 각각 하나의 거대한 기능들로 보고 그 안에 들어가는 세부 업무들을 설계하는 방식으로 진행하였다. 우리가 집중을 한 부분들은 계약과 보상 부분이다.

계약의 경우 크게 상품 개발, U/W, 계약 관리가 존재했다.

상품 개발의 핵심은 상품을 설계하고 인가를 요청하는 과정이었다. 이를 위해 보험, 인가, 금융 감독원 클래스를 선정하였다.

보험 클래스의 경우 상품을 설계하고 등록하는 부분을 담당한다. 해당 속성으로는 보험을 표현할 수 있는 가입 금액, 보험료, 보험 기간, 보험의 상품 코드, 만기 시 환급금 등이 존재한다고 생각하였다.

상품 인가의 경우 금융 감독원과의 연동이 필요하기에 금융감독원 클래스를 별도로 구성하였다. API 연동을 통해 인수 결과를 보내고 받는 역할을 넣을 예정이다. 금융감독원에게 상품을 **인가 요청**하기 위해서는 근거 자료들이 필요하다. 이를 위한 어디에 인가요청을 제출하였고, 요청을 한 사유와, 제품의 상세 내용 등을 속성으로 추가하고, 상품 인가를 처리하기 위한 상태들 즉, 요청 일자, 요청 번호, 승인 여부 같은 속성들을 관리하기위해 인가 클래스를 구성하였다.

U/W의 핵심은 위험을 평가해서 보험을 받을지 말지 결정하는 것이다.

이를 평가하기 위해서는 심사 과정이 필요하다. 그리고 심사를 하기 위해서는 청약 정보가 필요하다. 그렇기에 **심사** 와 청약 클래스를 가장 먼저 선정하게 되었고, 심사 과정 중에서 어떠한 내용들을 다룰지를 고민한 후 속성에 집어넣었다.

심사를 진행하였으면 그 결과가 기록되어야 하므로 **심사 결과** 클래스를 추가하였으며, 그러한 결과들을 결국에 사용자 데이터에 기입을 해야 하기 때문에 이를 위한 **사고이력** 클래스를 추가하였다.

청약 클래스에서는 기본적으로 청약에 필요한 속성들을 넣었고 추가적으로 심사를 위한 정보들이 필요하기에 속성들을 추가하였다.

보험사는 보험을 받을지 말지 결정하는 과정 중 공동 인수와 재보험 처리를 할지 판단할 수 있다. 보험금 규모가 너무 크면 공동 인수 처리를, 리스크가 크거나 손실 가능성이 높을 경우에는 재보험 처리를 해야 한다. 이를 위한 **공동인수** 클래스와 **재보험 처리** 클래스를 선

정하였다. 공동인수 클래스에서는 공동 인수를 위한 속성들을 고민했고 그렇게 나온 결과 어느 보험사와 공동인수를 할 것인지에 대해 관리하는 **공동 인수사** 클래스도 추가하였다. 재보험도 마찬가지로 재보험 판단에 필요한 정보들을 속성으로 집어넣었다.

U/W이력 클래스는 보험 인수 심사 과정에서 발생하는 판단 결과를 기록하고 관리하기 위해 선정하였다. 보험 인수 과정에서는 피보험자의 건강 상태, 직업, 나이, 과거 이력 등을 종합적으로 평가하여 계약을 승인하거나 거절하거나 조건을 변경하는 판단이 이루어지므로 별도의 이력 형태로 관리할 필요가 있다고 판단했다.

계약 관리의 핵심은 보유계약에 대해 평가 관리를 하는 것이다.

배서와 부활 관계는 다른 지급금관리, 분납/수금관리, 만기계약관리에 비해 위험 판단 필요가 상대적으로 높다. 그렇기에 U/W의 인수 판단이 다시 필요하기에 배서, 부활, 심사 클래스를 먼저 구성하였다.

계약관리는 이미 체결된 보험계약을 유지·변경·지급·종료 처리하는 업무이다. PDF에서 계약 관리는 분납/수금관리, 만기계약관리, 제지급금관리, 부활관리, 배서관리 등을 포함하고 있었고, 우리는 이 중 실제 시스템에서 상태 변경과 데이터 처리가 발생하는 업무들을 클래스로 선정하였다.

먼저 **계약** 클래스는 계약관리의 중심 객체이다. 분납/수금, 만기, 제지급금, 부활, 배서 모두 특정 계약을 기준으로 발생하기 때문에 계약을 기준 클래스로 두었다. 계약 클래스에는 계약번호, 납입주기, 납입여부, 지급금액, 계약상태 등 계약을 관리하는 데 필요한 속성을 배치하였다. 또한 계약을 조회하거나 갱신하고, 납입 여부나 지급금액을 관리하는 기능을 포함하였다.

배서 클래스는 기존 계약의 내용을 변경하는 업무를 표현하기 위해 선정하였다. 배서는 계약 내용 변경, 변경 사유, 변경 전후 내용 등을 관리해야 하며, 변경 내용에 따라 심사가 필요할 수 있다. 따라서 배서 유형, 변경 사유, 변경 전후 내용, 처리일시 등의 속성을 두고, 배서를 신청하고 처리하며 배서 내용을 반영하는 기능을 포함하였다.

부활 클래스는 실효된 계약을 다시 유효 상태로 되돌리는 업무를 표현하기 위해 선정하였다. 부활은 단순한 상태 변경이 아니라 미납보험료 납입 확인과 심사 과정을 거쳐야 하는 업무이므로 별도 클래스로 구성하였다. 부활 클래스에는 부활사유, 부활희망일, 심사결과, 처리일시 등의 속성을 두고, 부활신청, 부활심사, 부활처리 기능을 포함하였다.

심사 요청 클래스의 경우에는 해당 업무들을 판단할 기준들을 속성으로 구성하였다.

분납 클래스는 보험료 납입과 수금 처리를 관리하기 위해 선정하였다. 계약관리에서 보험료

납입 여부는 계약 유지 여부와 직접적으로 연결되며, 미납이 발생하면 계약 상태에 영향을 줄 수 있다. 따라서 미납보험료, 납입일자 등의 속성을 두고, 미납보험료를 계산하거나 납입처리 기능을 수행하도록 하였다. 분납과 관련된 보조 클래스로 이관 클래스와 미납안내 클래스를 추가하였다.

이관 클래스는 미납 상태가 일정 기준을 초과하거나 관리 부서가 변경되는 경우 해당 업무를 다른 담당자 또는 부서로 이관하기 위해 선정하였다.

미납안내 클래스는 보험료가 납입되지 않은 경우 고객에게 이를 안내하기 위해 선정하였다.

만기안내 클래스는 보험계약이 만기 시점에 도달했을 때, 고객에게 만기 정보를 전달하고 이후 처리 방향을 결정하기 위해 선정하였다. 만기안내 클래스에는 발송방법, 발송일시, 재계약의사, 확인일시 등의 속성을 넣어 고객에게 안내가 어떻게 전달되었는지, 고객이 재계약 의사를 언제 어떻게 표현했는지를 관리하도록 했다

제지급금 클래스는 계약과 관련하여 지급금이 발생하는 경우를 관리하기 위해 선정하였다. 지급 유형, 지급 금액, 지급일자 등을 속성으로 두고, 지급금을 산출하고 지급을 승인하며 지급 결과를 저장하는 기능을 포함하였다.

Account 클래스는 보험금 지급 및 자동이체 등 금융 거래를 처리하기 위한 계좌 정보를 관리하기 위해 선정하였다. 계좌번호, 계좌유형, 예금주, 은행명 등의 속성을 포함하도록 설계하였다. 또한 계좌 변경, 자동이체 처리, 잔액 확인 등의 기능을 포함하여 보험료 납입 및 보험금 지급 과정에서 필요한 금융 처리를 수행할 수 있도록 하였다.

피보험자 클래스는 보험의 보장 대상이 되는 고객의 정보를 관리하기 위해 선정하였다. 보험사는 피보험자의 연령, 직업, 건강 상태 등의 정보를 기반으로 위험을 평가하고 또한 피보험자 정보의 등록, 수정, 조회 기능을 포함하여 고객 정보 관리가 가능하도록 하였다.

보상의 경우 크게 보상 기획, 보상 처리가 존재했다.

보상 기획의 핵심은 보상을 평가, 관리하고 협력업체를 관리하는 것이다.

보상 평가 클래스의 경우 실적평가를 위한 기초자료를 수집/산출/분석/평가하는 것을 중심으로 구성하였다. 그를 위한 손해액, 평가 결과, 상태를 속성으로 구성하였다.

협력업체 클래스의 경우에는 협력체 이름, 연락처, 평가 상태 등을 구성해 평가, 관리, 등록을 할 수 있도록 구성하였다.

보상 처리의 핵심은 사고가 발생한 뒤, 사고를 접수하고 손해를 조사한 후 보험금 지급 여

부와 금액을 결정하여 지급·종결하는 것이다. 이에 따라 보상처리 영역에서는 사고접수, 손해조사, 보험금지급을 중심으로 클래스들을 구성하였다. 사고 접수와 가장 처음으로 시작되면서 추후에 프로세스들이 실행 되므로 가장 먼저 사고 접수 클래스를 선정하였다.

사고접수 클래스는 사고의 발생 시점, 사고 내용 등의 정보를 저장하며, 이후 손해조사 과정의 기준이 되는 핵심 객체로 사용된다.

손해조사 클래스는 접수된 사고에 대해 손해 여부와 손해 규모를 판단하는 과정을 표현하기 위해 선정하였다. 사고 관련 정보를 다룬다. 사고 접수나 손해 조사나 둘 다 결과에 대한 문서를 작성해야 할 필요가 있다.

이를 위해 **문서** 클래스를 별도로 추가하고 사고 접수와, 손해 조사에 대한 결과를 저장하여 관리하는 클래스를 추가하였다.

손해조사에서 외부 협력업체를 통한 조사 수행이 가능하므로, 이를 **조사위탁** 클래스를 별도로 분리하였다. 이를 통해 내부 조사와 외부 위탁을 구분하여 관리 가능하다. 또한 손해 조사에서 필요한 외부 협력 업체를 관리해야 하는데 이는 보상 기획에서 **협력 업체** 클래스를 생성했으므로 그것을 활용하기로 하였다.

보험금지급 클래스는 손해조사 결과를 기반으로 최종 지급 금액을 확정하고 실제 보험금지급을 처리하기 위해 선정하였다. 보험금지급은 지급금액, 지급계좌, 지급일시, 지급상태 등을 관리하며, 지급 완료 후 보상 사건을 종결하거나 후속 처리를 발생시키는 클래스이다.

이의제기 클래스는 보험금 지급 결과에 대해 보험가입자 또는 관련 당사자가 이의를 제기하는 경우를 관리하기 위해 선정하였다. 지급 금액이 부족하다고 판단되거나 지급 결과에 불만이 있는 경우 이의제기가 발생할 수 있으므로, 이의제기 사유, 기존 지급 내용, 조정 금액, 수용 여부 등을 속성으로 구성하였다.

구상처리 클래스는 보험금 지급 이후 제3자에게 책임이 있는 경우 보험사가 지급한 금액을 회수하는 업무를 표현하기 위해 선정하였다. 구상처리에서는 가해자 정보, 과실비율, 청구 금액, 입금계좌, 납부기한, 구상상태 등을 관리한다.

클래스 관계 선정 사유

Generalization 관계

보험(상위) - 자동차 보험, 화재, 해상 보험(하위)

"신동아화재는 1946 년 창립 이래 자동차, 장기, 해상, 화재, 특종보험을 중심으로 국내 중견 손해보험사로 영업을 해 왔습니다. " 해당 보험종목들은 모두 보험이라는 공통 개념을 가지며, 보험이라는 명사가 반복된다. 자동차·장기·해상·화재·특종이라는 구분에 따라 세부 유형이 나뉜다. 따라서 공통 속성과 기능은 보험 클래스에 두고, 각 보험종목은 보험을 상속받는 하위 클래스로 표현하여 Generalization 관계로 설정하였다.

문서(상위) - 사고 접수 서류, 지급 품의서(하위)

보상처리 과정에서는 사고접수와 손해조사, 보험금 지급 과정에서 다양한 형태의 문서가 생성된다. 이들 문서는 모두 생성일시, 문서 식별자, 문서 상태와 같은 공통적인 속성과 저장 및 임시 저장과 같은 공통 기능을 가진다. 사고접수서류와 지급 품의서는 각각 다른 목적과 세부 속성을 가지지만, 모두 문서라는 동일한 개념에 속하므로 상위 클래스인 Document 아래 하위 클래스인 Generalization 관계로 설정하였다.

Aggregation 관계

보험(전체) → 인가(부분)

보험은 독립적으로 존재할 수 있는 객체이지만, 인가요청은 반드시 특정 보험상품을 대상으로 생성된다. 즉, 인가요청은 보험이 존재해야 의미를 가지는 종속적인 객체이다. 또한 하나의 보험에 대해 여러 번의 인가요청이 발생할 수 있으며, 인가요청은 보험상품의 상태 변화(등록, 인가, 반려 등)에 직접적인 영향을 미친다. 이에 따라 인가요청은 보험에 포함된 처리 단위로 볼 수 있으므로, 보험을 전체 객체, 인가요청을 부분 객체로 보아 Aggregation 관계로 설정하였다.

사고접수(전체) → 손해조사(부분)

사고접수 클래스는 독립적으로 존재할 수 있는 객체이지만, 손해조사는 반드시 접수된 사고를 기준으로 수행된다. 즉, 손해조사는 사고접수가 존재해야 의미를 가지는 종속적인 객체이다. 또한 하나의 사고접수 건에 대해 손해조사가 진행되며, 손해조사 결과는 이후 보험

금 지급 여부와 금액 결정에 직접적인 영향을 미친다. 이에 따라 손해조사는 사고접수에 포함된 후속 처리 단위로 볼 수 있으므로, 사고접수를 전체 객체, 손해조사를 부분 객체로 보아 Aggregation 관계로 설정하였다.

손해조사(전체) → 위탁요청(부분)

손해조사 클래스는 독립적으로 존재할 수 있는 객체이지만, 위탁요청은 반드시 특정 손해조사를 기반으로 생성된다. 즉, 위탁요청은 손해조사가 존재해야 의미를 가지는 종속적인 객체이다. 또한 하나의 손해조사에 대해 여러 번의 위탁요청이 발생할 수 있으며, 위탁요청은 손해조사의 진행 과정과 결과에 직접적인 영향을 미친다. 이에 따라 위탁요청은 손해조사에 포함된 처리 단위로 볼 수 있으므로, 손해조사를 전체 객체, 위탁요청을 부분 객체로 보아 Aggregation 관계로 설정하였다.

손해조사(전체) → 보험금지급(부분)

손해조사 클래스는 독립적으로 존재할 수 있는 객체이지만, 보험금지급은 손해조사 결과를 기반으로 생성된다. 즉, 보험금지급은 손해조사가 존재해야 의미를 가지는 종속적인 객체이다. 또한 손해조사의 결과에 따라 보험금 지급 여부와 지급 금액이 결정되므로, 보험금지급은 손해조사의 결과를 반영하는 처리 단위이다. 이에 따라 보험금지급은 손해조사에 포함된 처리 결과 객체로 볼 수 있으므로, 손해조사를 전체 객체, 보험금지급을 부분 객체로 보아 Aggregation 관계로 설정하였다.

보험금지급(전체) → 이의제기(부분)

보험금지급 클래스는 독립적으로 존재할 수 있는 객체이지만, 이의제기는 반드시 특정 보험금지급 결과를 대상으로 발생한다. 즉, 이의제기는 보험금지급이 존재해야 의미를 가지는 종속적인 객체이다. 또한 하나의 보험금지급에 대해 여러 번의 이의제기가 발생할 수 있으며, 이의제기는 보험금 지급 결과에 대한 재검토와 수정에 직접적인 영향을 미친다. 이에 따라 이의제기는 보험금지급에 포함된 후속 처리 단위로 볼 수 있으므로, 보험금지급을 전체 객체, 이의제기를 부분 객체로 보아 Aggregation 관계로 설정하였다.

보험금지급(전체) → 구상처리(부분)

보험금지급 클래스는 독립적으로 존재할 수 있는 객체이지만, 구상처리는 보험금 지급 이후 제3자에게 책임이 있는 경우에 발생한다. 즉, 구상처리는 보험금지급이 존재해야 의미를

가지는 종속적인 객체이다. 또한 하나의 보험금지급에 대해 여러 건의 구상처리가 발생할 수 있으며, 구상처리는 지급된 금액 회수와 관련된 후속 처리에 직접적인 영향을 미친다. 이에 따라 구상처리는 보험금지급에 포함된 처리 단위로 볼 수 있으므로, 보험금지급을 전체 객체, 구상처리를 부분 객체로 보아 Aggregation 관계로 설정하였다.

구상처리(전체) → 이의제기(부분)

구상처리 클래스는 독립적으로 존재할 수 있는 객체이지만, 이의제기는 구상처리 과정에서 제3자가 구상 청구에 대해 불복하는 경우 발생할 수 있다. 즉, 해당 이의제기는 구상처리가 존재해야 의미를 가지는 종속적인 객체이다. 또한 구상처리 과정에서 발생한 이의제기는 구상금액, 과실비율, 납부 여부 등에 직접적인 영향을 미친다. 이에 따라 이의제기는 구상처리에 포함된 후속 처리 단위로 볼 수 있으므로, 구상처리를 전체 객체, 이의제기를 부분 객체로 보아 Aggregation 관계로 설정하였다.

배서(전체) → 심사요청(부분)

배서는 심사요청 없이도 독립적으로 존재할 수 있는 객체이다. 반면 심사요청은 반드시 특정 배서를 대상으로 생성되므로, 배서 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 배서에 대해 여러 번의 심사요청이 발생할 수 있으며, 심사요청의 결과가 배서의 처리 흐름에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 배서를 전체 객체, 심사요청을 부분 객체로 보는 Aggregation 관계로 설정하였다.

부활(전체) → 심사요청(부분)

부활은 심사요청 없이도 독립적으로 존재할 수 있는 객체이다. 반면 심사요청은 반드시 특정 부활을 대상으로 생성되므로, 부활 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 부활에 대해 여러 번의 심사요청이 발생할 수 있으며, 심사요청의 결과가 부활 처리의 승인 여부에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 부활을 전체객체, 심사요청을 부분객체로 보는 Aggregation 관계로 설정하였다.

계약(전체) → 배서(부분)

계약은 배서 없이도 독립적으로 존재할 수 있는 객체이다. 반면 배서는 반드시 특정 계약을 대상으로 생성되므로, 계약 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 계약에 대해 여러 번의 배서가 발생할 수 있으며, 배서는 계약의 내용 변경(특약 추가, 보

험료 변경 등)에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 계약을 전체객체, 배서를 부분객체로 보는 Aggregation 관계로 설정하였다.

계약(전체) → 부활(부분)

계약은 부활 없이도 독립적으로 존재할 수 있는 객체이다. 반면 부활은 반드시 특정 계약을 대상으로 생성되므로, 계약 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 계약에 대해 여러 번의 부활이 발생할 수 있으며, 부활은 실효된 계약의 효력 회복에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 계약을 전체객체, 부활을 부분객체로 보는 Aggregation 관계로 설정하였다.

계약(전체) → 분납/수금(부분)

계약 클래스는 독립적으로 존재할 수 있는 객체이지만, 분납/수금은 반드시 특정 계약을 기준으로 발생한다. 즉, 분납/수금은 계약이 존재해야 의미를 가지는 종속적인 객체이다. 또한 하나의 계약에 대해 여러 번의 납입과 수금 처리가 발생할 수 있으며, 납입 여부와 미납 상태는 계약 상태 변화에 직접적인 영향을 미친다. 이에 따라 분납/수금은 계약에 포함된 유지관리 처리 단위로 볼 수 있으므로, 계약을 전체 객체, 분납/수금을 부분 객체로 보아 Aggregation 관계로 설정하였다.

계약(전체) → 보상평가(부분)

계약 클래스는 독립적으로 존재할 수 있는 객체이지만, 보상평가는 특정 계약의 보상 처리 결과를 기준으로 수행된다. 즉, 보상평가는 계약 및 해당 계약에서 발생한 보상 이력이 존재해야 의미를 가지는 종속적인 객체이다. 또한 보상평가 결과는 손해액, 보상통계, 평가결과 등을 통해 계약 및 보상 업무의 관리에 직접적인 영향을 미친다. 이에 따라 보상평가는 계약과 관련된 보상관리 처리 단위로 볼 수 있으므로, 계약을 전체 객체, 보상평가를 부분 객체로 보아 Aggregation 관계로 설정하였다.

분납/수금(전체) → 이관(부분)

분납/수금 클래스는 독립적으로 존재할 수 있는 객체이지만, 이관은 분납/수금 과정에서 미납 상태가 지속되거나 담당 업무가 변경되는 경우에 발생한다. 즉, 이관은 분납/수금 처리가 존재해야 의미를 가지는 종속적인 객체이다. 또한 이관 처리는 미납 계약의 담당자 변경이나 후속 수금 방식에 직접적인 영향을 미친다. 이에 따라 이관은 분납/수금에 포함된

후속 처리 단위로 볼 수 있으므로, 분납/수금을 전체 객체, 이관을 부분 객체로 보아 Aggregation 관계로 설정하였다.

분납/수금(전체) → 미납안내(부분)

분납/수금 클래스는 독립적으로 존재할 수 있는 객체이지만, 미납안내는 납입기한 내 보험료가 납입되지 않은 경우에 발생한다. 즉, 미납안내는 분납/수금 처리가 존재해야 의미를 가지는 종속적인 객체이다. 또한 미납안내는 고객에게 미납금액과 납입방법을 안내하여 계약 유지 여부와 미납 상태 처리에 직접적인 영향을 미친다. 이에 따라 미납안내는 분납/수금에 포함된 후속 처리 단위로 볼 수 있으므로, 분납/수금을 전체 객체, 미납안내를 부분 객체로 보아 Aggregation 관계로 설정하였다.

계약(전체) → 지급(부분)

계약은 지급 없이도 독립적으로 존재할 수 있는 객체이다. 반면 지급은 반드시 특정 계약을 대상으로 발생하므로 계약 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 계약에 대해 여러 번의 지급이 발생할 수 있으며, 지급은 계약의 보험금 처리에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 계약을 전체 객체, 지급을 부분 객체로 보는 Aggregation 관계로 설정하였다.

피보험자(전체) → 심사이력(부분)

피보험자는 심사이력 없이도 독립적으로 존재할 수 있는 객체이다. 반면 심사이력은 반드시 특정 피보험자를 대상으로 생성되므로 피보험자 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 피보험자에 대해 여러 번의 심사이력이 누적될 수 있으며, 심사이력은 피보험자의 건강 및 위험 상태를 기록하는 처리 단위로 기능한다. 따라서 피보험자를 전체 객체, 심사이력을 부분 객체로 보는 Aggregation 관계로 설정하였다.

계약(전체) → 청약(부분)

계약은 청약 없이도 독립적으로 존재할 수 있는 객체이다. 반면 청약은 반드시 특정 계약의 생성을 목적으로 발생하므로 계약 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 계약에 대해 청약이 선행되어야 하며, 청약의 처리 결과가 계약의 성립 여부에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 계약을 전체 객체, 청약을 부분 객체로 보는 Aggregation 관계로 설정하였다.

계약(전체) → 재보험(부분)

계약은 재보험 없이도 독립적으로 존재할 수 있는 객체이다. 반면 재보험은 반드시 특정 계약을 기반으로 생성되므로 계약 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 계약에 대해 여러 재보험 처리가 발생할 수 있으며, 재보험은 계약의 위험 분산 처리에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 계약을 전체 객체, 재보험을 부분 객체로 보는 Aggregation 관계로 설정하였다.

청약(전체) → 심사(부분)

청약은 심사 없이도 독립적으로 존재할 수 있는 객체이다. 반면 심사는 반드시 특정 청약을 대상으로 수행되므로 청약 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 청약에 대해 여러 번의 심사가 진행될 수 있으며, 심사 결과는 청약의 승인 여부에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 청약을 전체 객체, 심사를 부분 객체로 보는 Aggregation 관계로 설정하였다.

청약(전체) → 공동보험(부분)

청약은 공동보험 없이도 독립적으로 존재할 수 있는 객체이다. 반면 공동보험은 반드시 특정 청약을 기반으로 생성되므로 청약 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 청약에 대해 공동보험 처리가 발생할 수 있으며, 공동보험은 청약의 위험 분담 처리에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 청약을 전체 객체, 공동보험을 부분 객체로 보는 Aggregation 관계로 설정하였다.

공동보험(전체) → 공동보험사(부분)

공동보험은 공동보험사 없이도 독립적으로 존재할 수 있는 객체이다. 반면 공동보험사는 반드시 특정 공동보험 계약을 기반으로 참여하므로 공동보험 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 공동보험에 대해 여러 공동보험사가 참여할 수 있으며, 공동보험사는 보험료 분담 및 위험 배분 처리에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 공동보험을 전체 객체, 공동보험사를 부분 객체로 보는 Aggregation 관계로 설정하였다.

심사(전체) → 심사결과(부분)

심사는 심사결과 없이도 독립적으로 존재할 수 있는 객체이다. 반면 심사결과는 반드시 특정 심사를 대상으로 생성되므로 심사 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 심사에 대해 심사결과가 도출되며, 심사결과는 청약의 승인·반려 여부에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 심사를 전체 객체, 심사결과를 부분 객체로 보는 Aggregation 관계로 설정하였다.

심사결과(전체) → 사고이력(부분)

심사결과는 사고이력 없이도 독립적으로 존재할 수 있는 객체이다. 반면 사고이력은 반드시 특정 심사결과와 연관된 피보험자를 대상으로 생성되므로 심사결과 없이는 존재 의미를 가지지 않는 종속적인 객체이다. 하나의 심사결과에 대해 여러 사고이력이 참조될 수 있으며, 사고이력은 심사결과의 위험도 평가에 직접적인 영향을 미치는 처리 단위로 기능한다. 따라서 심사결과를 전체 객체, 사고이력을 부분 객체로 보는 Aggregation 관계로 설정하였다.

V. 결론

본 보고서에서는 신동아화재 차세대 시스템 구축 FPL을 기반으로 유스케이스 다이어그램, 유스케이스 시나리오, 클래스 다이어그램의 세 단계로 시스템을 분석하고 설계하였다. 유스케이스 다이어그램을 통해 액터와 시스템 기능 간의 상호작용을 도출하였으며, 유스케이스 시나리오를 통해 각 기능의 Basic Flow, Alternate Flow, Exception Flow를 명세하였다. 이를 바탕으로 클래스 다이어그램을 작성하여 시스템 내부의 데이터 구조와 클래스 간의 관계를 표현하였다. 설계 결과로 계약 도메인, 청약 도메인, 사고처리 도메인으로 구분되는 클래스 구조를 도출하였으며, 도메인 간 연관관계를 통해 실제 보험 업무 흐름을 시스템적으로 표현하였다.

설계 과정에서는 몇 가지 어려움이 있었다. 먼저 유스케이스 시나리오 작성 시, Basic Path에서 액터와 시스템의 행위가 자연스럽게 교차되도록 흐름을 구성하는 것이 쉽지 않았다. 이를 위해 실제 업무 흐름을 단계별로 세분화하고, 액터의 입력과 시스템의 처리를 명확히 구분함으로써 문제를 해결하였다. 다음으로 "확인한다", "검토한다"처럼 시스템이 실제로 수행하는 행위와 그렇지 않은 표현을 구분하는 기준을 정립하는 데 혼란이 있었다. 이는 시스템이 데이터를 저장하거나 출력하는 행위를 기준으로 삼아 해결하였다. 클래스 다이어그램에서는 Aggregation과 Association의 관계를 구분하는 것이 어려웠는데, 한 클래스가 다른 클래스의 생명주기에 종속되는지 여부를 판단 기준으로 정하여 명확히 하였다. 또한 클래스 수가 늘어남에 따라 관계선이 겹치지 않도록 배치하는 데도 어려움이 있었으며, 도메인별로 클래스를 그룹화하여 배치함으로써 가독성을 개선하였다.

향후에는 본 설계를 기반으로 실제 시스템 구현 단계로 나아갈 수 있다. MVC 패턴을 적용하여 Model, View, Controller 계층을 구성하고 자바 기반으로 각 클래스를 구현함으로써 실제 동작하는 보험사 관리 시스템으로 발전시킬 수 있을 것이다.